

CORTEX

Ground Source of Truth — Engineering Design Document

MVP Edition — Unified Implementation Handbook for the 4-Person, 5-Week Build

Scope statement. This is the single, frozen, ground-truth architecture for CORTEX. Every idea from prior architecture notes and addenda is integrated directly into the chapter it belongs to, so the whole team builds from one coherent description. If it isn't named in this document, it isn't part of the MVP.

ET AI Hackathon 2026 — Problem Statement #8

AI for Industrial Knowledge Intelligence: Unified Asset & Operations Brain

Team Insomniacs

Bhavishy Agrawal · Suparn Agrawal

Anshika Jha · Anarghya Das

July 2026

Version 2.1 — Ground Source of Truth (supersedes v1.0 Engineering Handbook, v2.0, and all standalone retrieval / reasoning addenda)

Document Control

Field	Value
Document title	CORTEX MVP — Ground Source of Truth (Unified Engineering Design Document)
Supersedes	<i>CORTEX MVP — Engineering Design Document v1.0; v2.0</i> ; the standalone <i>Intelligent Retrieval Architecture</i> and <i>Compositional Reasoning</i> addenda
Status	Frozen ground truth; the only document a team member should build from
Audience	All four team members; judges (architecture and demo chapters)
Team	Bhavishy Agrawal, Suparn Agrawal, Anshika Jha, Anarghya Das
Timeline	5 weeks, single hackathon sprint
Non-goals	Enterprise scalability, Kubernetes, microservices, millions of users, research novelty, fine-tuned models
Primary judging weight	Innovation 25% / Business Impact 25% / Technical Excellence 20% / Scalability 15% / UX 15%

Table 1: Document control summary

How to use this document

This is the complete, self-contained description of CORTEX as it will actually be built, organised as one continuous architecture rather than a base document plus patches. If a chapter here appears to conflict with anything a team member remembers from an earlier draft, this document wins. Where a team member wants to add a service, database, agent, or framework not already named in [Chapter 5](#) through [Chapter 18](#), the correct action is to record it in [Chapter 19](#) and keep building the MVP as specified here — not to build it now. [Figure 5.1](#) on the next few pages is the single picture that explains the whole system; every later chapter is detail on one box of that picture.

Contents

Document Control	i
1 Problem Overview	1
1.1 The problem in one paragraph	1
1.2 Root causes CORTEX addresses	1
1.3 Why a knowledge graph, not just a chatbot	1
1.4 Scope boundary for this document	2
1.5 Stakeholders and their concerns	2
2 Product Vision	3
2.1 Vision statement	3
2.2 The five pillars, and what they mean at implementation level	4
2.3 Why CORTEX differs from conventional RAG	4
2.4 Product narrative	5
2.5 Design principles	5
2.6 What “done” looks like for the MVP	6
3 Functional Requirements	7
3.1 Requirement notation	7
3.2 Ingestion & Extraction	7
3.3 Knowledge Graph & Facts	8
3.4 Retrieval & Copilot (CORTEX Ask)	9
3.5 Agents	10
3.6 Frontend	11
3.7 Auth & Audit	11
4 Non-functional Requirements	12
4.1 Scale envelope (deliberately small)	12
4.2 Latency targets	13
4.3 Reliability & degradation	14
4.4 Explainability & trust	14
4.5 Security (demo-appropriate, not enterprise)	14
4.6 Maintainability	14
4.7 Portability	15
4.8 Usability	15
5 System Architecture	16
5.1 The whole system in one diagram	16
5.2 Key technology decisions at a glance	17
5.3 Three-layer architecture	17
5.4 Layer 1: Presentation (Next.js)	18

5.5	Layer 2: Fabric + Retrieval + Agents (one FastAPI app)	18
5.6	Layer 3: Ingestion + Extraction (background worker)	19
5.7	Data stores	19
5.8	Request lifecycle: Copilot query	19
5.9	Component interaction summary	21
5.10	Demo corpus target	21
6	Data Flow	22
6.1	End-to-end pipeline	22
6.2	Stage 1: Document Upload	22
6.3	Stage 2: Parsing (Docling)	23
6.4	Stage 3: Entity & Specification Extraction	23
6.5	Stage 4: Entity Resolution	24
6.6	Stage 5: Knowledge Graph	24
6.7	Stage 6: Vector DB	25
6.8	Stage 7: Context-Aware Retrieval	25
6.9	Stage 8: Copilot	25
6.10	Stage 9: Agents (including Compositional Reasoning)	25
6.11	Stage 10: Frontend	25
6.12	Pipeline-wide error handling policy	26
7	Database Design	27
7.1	Overview	27
7.2	Entity-relationship overview	27
7.3	Table: <code>document_registry</code>	27
7.4	Table: <code>facts</code>	28
7.5	Table: <code>entity_registry</code> and <code>entity_aliases</code>	28
7.6	Table: <code>users</code>	29
7.7	Table: <code>audit_log</code>	29
7.8	Lexical search	30
7.9	Migrations	30
8	Knowledge Graph	31
8.1	Purpose and responsibilities	31
8.2	Ontology	31
8.2.1	Extended ontology for compositional reasoning	32
8.3	Neo4j node and edge model	32
8.4	Graph materialisation (write path)	33
8.5	Graph traversal (read path)	33
8.5.1	Fixed-depth traversal (shallow mode)	33
8.5.2	Adaptive, context-aware traversal (full mode)	34
8.6	Confidence model on the graph	34
8.7	Graph updates and refresh	34
8.8	Edge cases	35
8.9	Testing	35
8.10	Expected latency	35
8.11	Dependencies	35
8.12	Future extension	35
9	Retrieval	36
9.1	Purpose and responsibilities	36
9.1.1	Why a fixed hop count was not enough	36
9.2	Chunking	37

9.3	Query classification	37
9.4	Context Assembly	37
9.5	Seed Expansion	38
9.6	Adaptive Traversal	39
9.7	Structural Enrichment	40
9.8	Passage Synthesis	41
9.9	Fusion weights	42
9.10	The three pathways	42
9.11	Merge and re-ranking	42
9.12	Generation and citation	43
9.13	Interfaces	43
9.14	Retained fast path: shallow traversal mode	43
9.15	Edge cases	43
9.16	Testing	44
9.17	Performance budget	44
9.18	What this does NOT require	44
9.19	Dependencies	45
9.20	Future extension	45
10	AI Models	46
10.1	Two-tier routing	46
10.2	Why every model exists	47
10.3	Expected input/output per task	47
10.4	Prompt strategy	47
10.5	JSON schema example: bulk extraction (extended)	48
10.6	Does compositional reasoning need fine-tuning?	48
10.7	Failure handling	49
10.8	Cost and latency management	49
10.9	Dependencies	49
10.10	Future extension	49
11	Agent Design	50
11.1	Pattern: LangGraph Supervisor → three workers	50
11.2	Table of workers	51
11.3	Supervisor: responsibilities and decision flow	51
11.4	Autonomy tiers	52
11.5	CORTEX Asset	52
11.5.1	Four capabilities, one worker	52
11.5.2	Prompt strategy for capabilities 1–2 (QA/history)	52
11.5.3	Tools	52
11.5.4	Compositional Reasoning: the Constraint-Gather-Filter-Reason (CGFR) pattern	53
11.5.5	Agent prompt addition for CGFR	55
11.5.6	Edge case handling	55
11.6	CORTEX Diagnose	55
11.7	CORTEX Comply	56
11.8	The enhanced context_graph_query tool	56
11.9	Prompts (representative)	56
11.10	Interfaces	57
11.11	Logging	57
11.12	Testing	57
11.13	Expected latency	57

11.14	Dependencies	57
11.15	Future extension	58
12	APIs	59
12.1	Endpoint surface	59
12.2	Request/response examples	59
12.2.1	POST /api/documents	59
12.2.2	POST /api/query	60
12.2.3	GET /api/entities/{tag}/graph?depth=2	60
12.2.4	POST /api/entities/{tag}/feedback	60
12.2.5	POST /api/agents/diagnose	60
12.2.6	GET /api/agents/diagnose/{job_id}	61
12.2.7	GET /api/agents/comply/gaps	61
12.2.8	POST /api/agents/asset/recommend (compositional reasoning)	61
12.3	Error format	61
12.4	Streaming transport	62
12.5	Versioning	62
12.6	Rate limiting and auth	62
12.7	Testing	62
13	Frontend	63
13.1	Graph-first navigation	63
13.2	Page-by-page	64
13.3	Component hierarchy	64
13.4	State management	65
13.5	Streaming UI	65
13.6	Graph animations	65
13.7	Replay Reasoning	65
13.8	Compositional comparison tables	65
13.9	Compliance, made visual	65
13.10	Mobile responsiveness	66
13.11	Testing	66
13.12	Expected latency	66
13.13	Dependencies	66
13.14	Future extension	66
14	Authentication	67
14.1	Purpose and scope	67
14.2	Roles	67
14.3	JWT flow	67
14.4	Permissions matrix	68
14.5	Edge cases	68
14.6	Testing	68
14.7	Dependencies	68
14.8	Future extension	68
15	Evaluation	69
15.1	Philosophy	69
15.2	Evaluation dashboard	70
15.3	Ground truth creation	70
15.4	Manual evaluation process	71
15.5	Dashboard implementation	71
15.6	Expected targets	71

15.7 Dependencies	71
15.8 Future extension	71
16 Testing	72
16.1 Testing philosophy	72
16.2 Unit testing	72
16.3 Integration testing	73
16.4 End-to-end (UI) testing	73
16.5 AI-specific testing	73
16.6 Test data	73
16.7 What is explicitly not tested	73
16.8 Dependencies	74
17 Deployment	75
17.1 Target	75
17.2 Docker Compose services	75
17.3 Folder structure	76
17.4 Environment variables	77
17.5 Startup sequence	77
17.6 Configuration management	77
17.7 Observability	77
17.8 Degradation plan for the demo	77
17.9 Dependencies	78
17.10Future extension	78
18 Demo	79
18.1 Why one continuous interaction	79
18.2 Complete demo script — “A Day in the Life”	80
18.3 Transitions	80
18.4 Judge questions and expected answers	81
18.5 Failure recovery during the demo	82
18.6 Rehearsal plan	82
19 Future Enterprise Roadmap	83
19.1 Principle	83
19.2 MVP → enterprise upgrade path	84
19.3 Sequencing if the project continues past the hackathon	85
19.4 Closing	85
Appendix: Decision Log	86

List of Tables

1	Document control summary	i
1.1	Stakeholder concerns and the chapters that satisfy them	2
2.1	The five CORTEX pillars mapped to implementation	4
2.2	CORTEX’s four differentiators versus conventional RAG	5
3.1	Ingestion & extraction functional requirements	7
3.2	Knowledge graph functional requirements	8
3.3	Retrieval and copilot functional requirements	9
3.4	Agent functional requirements	10
3.5	Frontend functional requirements	11
3.6	Auth and audit functional requirements	11
4.1	Scale envelope	12
4.2	Latency targets, measured per Chapter 15	13
5.1	Key technology decisions at a glance	17
5.2	Data stores and their roles	19
5.3	Component interaction summary	21
6.1	Pipeline-wide error handling policy	26
8.1	Core entity types	31
8.2	Core and extended relationships	32
9.1	Failure modes of a fixed-depth, query-only graph traversal	36
9.2	Inputs to context assembly	37
9.3	Key design decisions in adaptive traversal	40
9.4	When structural enrichment runs	41
9.5	Query-type → pathway weight lookup table	42
9.6	The three retrieval pathways	42
9.7	Context-aware retrieval performance budget	44
9.8	Deliberately excluded approaches, and their roadmap trigger	44
10.1	Task-to-model mapping	46
10.2	Expected input/output shape per task	47
10.3	Zero-shot readiness of compositional reasoning capabilities	48
10.4	Where fine-tuning would help post-MVP, and the MVP alternative used instead	49
11.1	The three CORTEX workers	51
11.2	Two autonomy tiers, no policy-engine microservice	52
11.3	CGFR edge case handling	55

13.1	Page-by-page breakdown	64
14.1	The four seeded demo roles	67
14.2	Coarse RBAC permissions matrix	68
15.1	Evaluation dashboard metrics	70
15.2	Expected evaluation targets	71
16.1	Unit test coverage by area	72
17.1	Environment variables (<code>.env</code>)	77
18.1	Demo script with timing and speaking points	80
18.2	Anticipated judge questions and rehearsed answers	81
18.3	Failure recovery playbook	82
19.1	MVP to enterprise upgrade path	84
19.2	Architectural decision log	87

List of Figures

5.1	Master pipeline: the entire CORTEX system in one diagram. Top: ingestion (Dolcling parse → extraction → resolution → Neo4j/Qdrant/Postgres). Middle: a query is classified, its context assembled (query + chat history + focused graph node), and fanned out across adaptive graph traversal, vector, and lexical retrieval. Bottom: fused evidence feeds generation and, where relevant, an agent (including compositional CGFR reasoning), both landing back in the UI as a cited answer or a live graph update.	16
5.2	Three-layer architecture, deployed as four Docker Compose service groups — unchanged by the context-aware retrieval and compositional reasoning capabilities . .	18
5.3	Sequence diagram: Copilot query through context assembly, retrieval, generation, and optional agent follow-up (including compositional reasoning) — the same flow as Figure 5.1, shown as a timeline instead of a pipeline	20
6.1	End-to-end data flow — the same ten stages as Figure 5.1, laid out linearly	22
7.1	Postgres entity-relationship overview	27
11.1	Supervisor–worker agent architecture — unchanged shape; CORTEX Asset’s tool set is extended, no fourth worker is introduced	50
11.2	The four-step Constraint-Gather-Filter-Reason (CGFR) pattern	54
13.1	Graph-first navigation loop, including the focused_tag feedback into retrieval . . .	63

Chapter 1

Problem Overview

1.1 The problem in one paragraph

Asset-intensive industrial organisations run on documents, not databases: P&IDs, work orders, OEM manuals, inspection reports, regulatory clauses, spare parts catalogs, and project files, each living in its own system, each readable only by whoever already knows where to look. Engineers spend a large share of every working day searching for information that already exists somewhere in the organisation rather than acting on it. CORTEX exists to close that gap: it ingests the heterogeneous document corpus of a plant, extracts and links the facts inside it into a single knowledge graph, and exposes that graph through a cited copilot and a small set of specialist agents, so that the answer to “why does Pump P-101A keep failing” takes seconds instead of an afternoon of cross-referencing binders — and so that the answer to a question no single document ever answered, like “which funnel should I use for pipe P-301”, can still be constructed, cited, and trusted.

1.2 Root causes CORTEX addresses

- **Fragmentation.** A single plant operates across many disconnected document systems (P&IDs, work orders, procedures, inspection records, spare parts catalogs), with no shared identifier space linking a tag number in one system to the same tag in another.
- **Silent knowledge loss.** Experienced engineers retire carrying undocumented operational judgement that never made it into a procedure or a manual.
- **Unactionable data.** Signals frequently exist (a sensor reading, a permit, a maintenance flag) but no intelligence layer connects them to a decision before the decision window closes.
- **Un-auditable answers.** Even where search tools exist, they return documents, not cited, trustable facts an engineer can act on without re-reading the source.
- **Answers that were never written down.** Some of the most valuable questions — “which spare part is compatible with this pipe” — have answers that exist only as scattered specifications across several documents that were never mentioned together in any single passage. No retrieval pathway that looks for an existing sentence will ever find them; they must be *constructed* from parts.

1.3 Why a knowledge graph, not just a chatbot

A plain RAG chatbot over a document corpus answers “what does the manual say about P-101A” reasonably well. It cannot answer “what else is connected to P-101A, what has failed

on it before, and what regulation applies to it” because those are graph-traversal questions, not passage-retrieval questions, and it cannot answer “which funnel fits this pipe” at all, because that is a graph-*construction* question with no matching passage anywhere. CORTEX’s defining bet is that industrial knowledge is fundamentally relational — equipment connects to equipment, failures recur on equipment types, regulations apply to equipment classes, specifications constrain compatible parts — and that a knowledge graph, explored adaptively and reasoned over compositionally, is the only structure that represents that relational shape directly, with vector and lexical retrieval layered on top for the passages that back each fact. [Chapter 2](#), §2.3 makes this comparison concrete against conventional RAG, and [Chapters 9](#) and [11](#) are where it is actually implemented.

1.4 Scope boundary for this document

This document describes the MVP exactly as it will be built for the 5-week hackathon window. It intentionally excludes any subsystem marked *Won’t (this cycle)* in [Chapter 19](#): Kafka, Kubernetes, a bitemporal claim store, ontology versioning, a DuckDB/ClickHouse analytical mirror, RT-DETR+SAM2 line tracing, three-tier autonomy with OPA, Keycloak, OT/historian integration, offline-first PWA/CRDT sync, Microsoft GraphRAG-style offline community summarisation, Neo4j GDS Enterprise, fine-tuned models, and a from-scratch academic benchmark. Every one of these has a named upgrade trigger in [Chapter 19](#); none of them is a week-1–5 deliverable.

1.5 Stakeholders and their concerns

Stakeholder	Primary concern	Where addressed
Maintenance Engineer	“What has failed on this equipment before, and why?”	CORTEX Asset (Chapter 11), Copilot (Chapter 9)
Reliability Engineer	“Which spare part is actually compatible with this equipment?”	CORTEX Asset compositional reasoning (Chapter 11 , §11.5.4)
Plant Manager	Single view of risk and compliance across the facility	Graph explorer, Compliance score (Chapter 13)
Compliance Officer	Evidence of regulatory conformance, gap visibility	CORTEX Comply (Chapter 11)
Reliability Engineer (RCA)	Root-cause hypotheses backed by evidence, not guesses	CORTEX Diagnose (Chapter 11)
Hackathon judge	A working, explainable, checkable demo in 5–8 minutes	Chapter 18

Table 1.1: Stakeholder concerns and the chapters that satisfy them

Chapter 2

Product Vision

2.1 Vision statement

Vision

CORTEX is an Industrial Intelligence Fabric that transforms fragmented engineering documents into a living, queryable knowledge graph — enabling explainable AI copilots and specialist agents to reason over industrial knowledge, not isolated documents, and to construct answers that no single document ever stated.

2.2 The five pillars, and what they mean at implementation level

Pillar	Product meaning	Implementation surface
Industrial Knowledge	Every document becomes structured, linked facts in one graph, including specification-level attributes.	Ingestion pipeline (Chapter 6), <code>facts</code> table, Neo4j projection (Chapter 8)
Industrial Reasoning	Specialist agents traverse the graph to answer cross-document questions.	CORTEX Asset / Diagnose / Comply (Chapter 11)
Contextual Retrieval	Retrieval understands conversational context and intent, not just the current query string, and explores the graph adaptively rather than at a fixed hop count.	Context-aware traversal pipeline (Chapter 9)
Compositional Reasoning	Where no document ever states the answer, CORTEX constructs it from scattered, cited specifications rather than saying “not found.”	CGFR reasoning pattern within CORTEX Asset (Chapter 11 , §11.5.4)
Industrial Trust & Explainability	Every fact carries a source, a confidence, and a status; every answer — retrieved or constructed — is cited to the passage or fact that produced it.	<code>facts.status</code> , <code>superseded_by</code> , audit log (Chapter 7); citation-resolution check (Chapter 9)

Table 2.1: The five CORTEX pillars mapped to implementation

2.3 Why CORTEX differs from conventional RAG

This is the single question a judge needs answered in under thirty seconds: what is actually novel here, beyond “another RAG chatbot”? The honest answer is four compounding differences, each of which closes a specific failure mode a plain RAG pipeline cannot.

Traditional RAG	CORTEX	What this unlocks
Passage retrieval only (vector similarity over chunks)	Hybrid retrieval: graph traversal + vector + lexical, fused by query type (Chapter 9)	Answers relational questions (“what else is connected to this”) that no single passage states
Fixed context: only the current query string is embedded and searched	Context-aware traversal: seeds assembled from query, chat history, and the focused graph node (Chapter 9, §9.4)	Handles implicit follow-ups (“what about the valve?”) with no named entity
Fixed-depth or single-hop graph lookups, if a graph exists at all	Adaptive, best-first, relevance-decayed traversal bounded by a node budget, not a hop count (Chapter 9, §9.6)	Surfaces distant but relevant nodes (a sibling pump 5+ hops away) that a fixed hop count would miss
Direct generation from retrieved passages; “not found” if no passage matches	Compositional reasoning (CGFR): decompose into constraints, gather candidates, filter, justify (Chapter 11, §11.5.4)	Answers questions no document ever stated, by constructing the answer from scattered, individually-cited specifications

Table 2.2: CORTEX’s four differentiators versus conventional RAG

The one-sentence pitch

Conventional RAG retrieves a passage that already answers the question; CORTEX traverses a graph that understands conversational context, and — when no passage ever answered the question at all — constructs and cites an answer from the facts that do exist.

2.4 Product narrative

The product narrative that drives every UI and demo decision is *graph-first*: a user does not open a document list, they open a living graph, click a node, and watch the system reveal everything connected to it — documents, history, failure patterns, agent reasoning, even parts that were never mentioned alongside it in any document — as a single continuous interaction rather than a sequence of disconnected screens. Chapter 13 and Chapter 18 are built entirely in service of this narrative; if a proposed feature does not make the graph feel more alive or more trustworthy, it does not belong in the MVP.

2.5 Design principles

1. **Cite or say nothing.** No generated sentence stands without a resolvable [doc_id:passage_id] tag or a graph fact_id; unsupported claims are rejected and regenerated, not shipped softened. Where a constructed answer draws on general engineering reasoning rather than a document, it is explicitly flagged as such (Chapter 11, §11.5.4).
2. **Never silently overwrite.** Conflicting facts both remain queryable, one marked **superseded** or **disputed**, never deleted.
3. **Retrieval follows intent, not just keywords.** The system assembles context from the current query, recent conversation, and the currently focused graph node before deciding what to retrieve (Chapter 9).

4. **Construct, don't just retrieve.** When an answer is derivable from facts that were never stated together in any passage, the system decomposes the question into constraints, gathers candidates, filters by compatibility, and cites every constituent fact ([Chapter 11](#), §11.5.4) — it does not give up because no single “golden passage” exists.
5. **Fewer moving parts, more demo time.** Every architectural choice in [Chapter 5](#) trades a small amount of theoretical scalability for a large amount of integration reliability under a 5-week clock. Context-aware traversal and compositional reasoning are implemented as application-level orchestration over the existing Neo4j, Qdrant, and Postgres stack — not as new services.
6. **Judge-checkable over judge-tellable.** A metric the team can run live ([Chapter 15](#)) beats a claim on a slide the team cannot defend under questioning.
7. **Graceful degradation, not silent failure.** Every pipeline stage ([Chapter 6](#)) has a defined fallback path so a single bad document, a single flaky model call, or an incomplete specification cannot take down the demo.

2.6 What “done” looks like for the MVP

The MVP is done when a single rehearsed, scripted path ([Chapter 18](#)) — click a piece of equipment, watch its context light up via context-aware traversal, invoke CORTEX Diagnose, watch it reason live with citations, ask CORTEX Asset a compositional “which part fits this” question and watch it construct and cite an answer, check the compliance score, submit a correction and watch the graph update on screen — runs reliably, end to end, against the real 100–300 document demo corpus, in under eight minutes, with no manual intervention.

Chapter 3

Functional Requirements

3.1 Requirement notation

Each requirement has an ID of the form **FR-*<area>-<n>***, a priority (Must / Should / Could), and a pointer to the chapter that implements it.

3.2 Ingestion & Extraction

ID	Requirement	Priority	Ref
FR-ING-1	System shall ingest PDF (native and scanned) and XLSX documents via Docling / pandas.	Must	Chapter 6
FR-ING-2	System shall extract per-element bounding boxes for every parsed element to support citation.	Must	Chapter 6
FR-ING-3	System shall extract the core entity types (Equipment, FailureMode, MaintenanceAction, Procedure, Regulation, Document, Personnel, SparePart) and their relationships.	Must	Chapter 8
FR-ING-4	System shall resolve entity mentions to canonical tags via the 2.5-stage resolution pipeline (exact, fuzzy, embedding fallback).	Must	Chapter 8
FR-ING-5	System shall process P&ID images via a multi-modal VLM call to extract equipment symbols and approximate adjacency as graph edges.	Should	Chapter 6
FR-ING-6	Unresolved entity mentions shall be logged, not silently dropped.	Must	Chapter 8
FR-ING-7	System shall extract specification-level facts (dimensions, material, operating temperature, operating pressure, compatibility ranges) using the extended predicate set, in addition to relational facts.	Must	Chapter 6, Chapter 8

Table 3.1: Ingestion & extraction functional requirements

3.3 Knowledge Graph & Facts

ID	Requirement	Priority	Ref
FR-KG-1	Every extracted fact shall be written to the facts table with source, confidence, extraction method, and status.	Must	Chapter 7
FR-KG-2	Only active facts shall be projected into Neo4j; superseded / disputed facts remain queryable in Postgres.	Must	Chapter 8
FR-KG-3	A human correction submitted via the feedback endpoint shall create a new fact and mark the prior one superseded , never delete it.	Should	Chapter 12
FR-KG-4	The graph shall support adaptive, relevance-decayed traversal (not a fixed hop count) for context-aware retrieval, in addition to a fast fixed-depth (≤ 2) mode.	Must	Chapter 8 , Chapter 9
FR-KG-5	The graph schema shall support specification-type relationships (hasSpec , hasMaterial , operatingTemp , operatingPressure , fitsRange , compatibleWith , ratedFor) without any schema migration, relying on Neo4j's schema-free relationship types.	Must	Chapter 8

Table 3.2: Knowledge graph functional requirements

3.4 Retrieval & Copilot (CORTEX Ask)

ID	Requirement	Priority	Ref
FR-RAG-1	System shall classify each incoming query into factual / diagnostic / procedural / open.	Must	Chapter 9
FR-RAG-2	System shall fan out to graph, vector, and lexical retrieval in parallel and fuse results by query-type weights.	Must	Chapter 9
FR-RAG-3	System shall re-rank fused candidates with a cross-encoder to 5–8 chunks before generation.	Must	Chapter 9
FR-RAG-4	Every generated sentence shall carry a <code>[doc_id:passage_id]</code> citation tag resolvable to a retrieved passage; unresolved tags trigger one regeneration attempt.	Must	Chapter 9
FR-RAG-5	Copilot responses shall stream over SSE/WebSocket, not block on full generation.	Must	Chapter 12
FR-RAG-6	The graph retrieval pathway shall assemble seed context from the current query, recent chat history, and the currently focused graph node (not from explicit query entities alone).	Must	Chapter 9
FR-RAG-7	The graph retrieval pathway shall expand seeds via embedding similarity and equipment-type matching, in addition to direct entity mentions.	Must	Chapter 9
FR-RAG-8	Graph traversal shall use a best-first, relevance-decaying algorithm bounded by a node budget and a depth ceiling, rather than a fixed hop count, for diagnostic and open queries.	Must	Chapter 9
FR-RAG-9	For diagnostic and open queries, the retrieval pathway shall additionally compute multi-seed shortest paths and lightweight hub detection to surface cross-domain bridge nodes.	Should	Chapter 9
FR-RAG-10	A fast, fixed-depth (“shallow”) traversal mode shall remain available for factual queries and as a fallback.	Must	Chapter 9, §9.14

Table 3.3: Retrieval and copilot functional requirements

3.5 Agents

ID	Requirement	Priority	Ref
FR-AGT-1	A Supervisor shall route queries and explicit invocations to exactly one of three workers: CORTEX Asset, CORTEX Diagnose, CORTEX Comply.	Must	Chapter 11
FR-AGT-2	CORTEX Asset shall surface recurring failure modes and pattern-matched past incidents for a given tag.	Must	Chapter 11
FR-AGT-3	CORTEX Diagnose shall generate ranked, evidence-cited causal hypotheses and discard self-critiqued unsupported ones.	Must	Chapter 11
FR-AGT-4	CORTEX Comply shall check a curated set of 10–15 regulation clauses against equipment records and flag gaps.	Must	Chapter 11
FR-AGT-5	Every worker shall be capped at a fixed maximum tool-call count to prevent runaway loops.	Must	Chapter 11
FR-AGT-6	Tier-2 (propose) actions shall render with a confirm/dismiss control gated by role; no action shall auto-execute.	Should	Chapter 11
FR-AGT-7	Agent reasoning steps shall stream live and be replayable after completion.	Should	Chapter 13
FR-AGT-8	CORTEX Asset shall support compositional reasoning for component-selection and compatibility questions (“which X for Y”) via the Constraint-Gather-Filter-Reason pattern, without changing the overall supervisor–worker architecture.	Must	Chapter 11, §11.5.4
FR-AGT-9	When a compositional answer cannot be fully grounded in retrieved facts, CORTEX Asset shall explicitly flag which constraint is unverified or which recommendation rests on general engineering knowledge rather than a document.	Must	Chapter 11, §11.5.4

Table 3.4: Agent functional requirements

3.6 Frontend

ID	Requirement	Priority	Ref
FR-UI-1	Dashboard shall open directly on the graph explorer, not a document list.	Must	Chapter 13
FR-UI-2	Search shall always resolve to a graph node and re-centre the view on it.	Must	Chapter 13
FR-UI-3	Node click shall open a side panel with CORTEX Ask, maintenance timeline, and linked documents, and shall send the clicked node's tag to the backend as <code>focused_tag</code> on subsequent queries.	Must	Chapter 13, Chapter 9
FR-UI-4	Compliance shall render as a single score with red/amber/green drill-down, not a table.	Should	Chapter 13
FR-UI-5	UI shall be responsive on a phone browser; a voice-input button is a stretch goal.	Should/Could	Chapter 13
FR-UI-6	CORTEX Asset compositional-reasoning answers shall render candidate comparisons as a structured table (property-by-property pass/fail), not as unstructured prose.	Should	Chapter 13

Table 3.5: Frontend functional requirements

3.7 Auth & Audit

ID	Requirement	Priority	Ref
FR-AUTH-1	System shall support 4 seeded roles (technician, engineer, compliance officer, admin) with route-level RBAC.	Must	Chapter 14
FR-AUTH-2	Every agent action and query shall be written to <code>audit_log</code> with actor, action, payload, timestamp.	Must	Chapter 7

Table 3.6: Auth and audit functional requirements

Chapter 4

Non-functional Requirements

4.1 Scale envelope (deliberately small)

Every NFR below is scoped to the demo envelope — 100–300 documents, 15–25 pieces of equipment, 2–3 P&IDs, a handful of concurrent demo users. These are not placeholders to be grown later inside this build; they are the actual target. Designing past them (connection pools for thousands of concurrent users, sharded graph storage, full GDS clustering, etc.) is explicitly out of scope and is recorded instead in [Chapter 19](#).

Dimension	Target
Corpus size	100–300 documents, mixed PDF (native + scanned) and XLSX, 2–3 P&ID images
Equipment count	15–25 tags, enough overlapping records for a connected graph
Concurrent users	Single demo machine / single cloud VM; 1–5 concurrent sessions
Query volume	Low tens of queries per demo run; no sustained load
Ingestion volume	Full corpus ingested once during Weeks 1–2, incremental additions thereafter

Table 4.1: Scale envelope

4.2 Latency targets

Operation	Target (P95)	Notes
Copilot first token	< 2s	Time to first SSE token after <code>/api/query</code>
Copilot full answer	< 8s	Includes retrieval fan-out, re-rank, generation, citation check
Graph retrieval pathway (full, diagnostic/open)	< 500ms	Five-phase context-aware pipeline: context assembly, seed expansion, adaptive traversal, structural enrichment, passage synthesis (Chapter 9)
Graph retrieval pathway (shallow, factual)	< 300ms	Fixed-depth ≤ 2 fast path, retained as §9.14
Vector search (top 20)	< 200ms	Qdrant, single collection, demo-scale
Lexical search (top 20)	< 150ms	Postgres FTS / BM25
Agent invocation (Asset/Comply, non-compositional)	< 15s	Single graph query + one LLM synthesis call
Agent invocation (Asset, compositional/CGFR)	< 25s	Constraint extraction + two parallel retrievals + compatibility filtering + justified selection (Chapter 11, §11.5.4)
Agent invocation (Diagnose)	< 30s	Plan-then-execute with self-critique pass
Document ingestion (per doc)	< 60s	Docling parse + extraction + resolution, mid-size PDF
P&ID VLM extraction	< 20s per drawing	Single multimodal call

Table 4.2: Latency targets, measured per Chapter 15

4.3 Reliability & degradation

The system has no uptime SLA (it is a single demo run), but must degrade gracefully rather than fail visibly on stage:

- If P&ID live VLM extraction is slow or fails during the demo, fall back to 2–3 pre-processed drawings with graph edges pre-loaded (see [Chapter 17](#)).
- If a single document fails to parse, ingestion continues for the remaining corpus; the failure is logged, not fatal to the batch.
- If the frontier model call for Copilot generation times out, retry once with the same retrieved context before surfacing an error state to the user.
- If an agent exceeds its `max_iterations` cap, it returns its best partial answer with an explicit “inconclusive” marker rather than hanging.
- If context-aware traversal’s structural-enrichment phase ([Chapter 9](#)) is slow, it is dropped and the answer proceeds on adaptive-traversal results alone, per the per-phase budget in [Chapter 9](#).
- If a compositional (CGFR) query cannot find candidate specifications for one side of the comparison, CORTEX Asset returns a partial, explicitly-flagged answer rather than fabricating a specification ([Chapter 11](#), §11.5.4).

4.4 Explainability & trust

- Every Copilot sentence must resolve to a specific `doc_id` and `passage_id`, or, for graph-derived synthetic passages, to a specific `fact_id`.
- Every fact must carry `source_doc_id`, `confidence`, and `status`.
- Every agent conclusion, including compositional (CGFR) recommendations, must be traceable to the graph nodes/edges and documents that produced it (evidence chain, replayable).
- Any part of a constructed answer that is not grounded in a retrieved fact must be explicitly labelled as general engineering reasoning, never presented as if it were sourced.

4.5 Security (demo-appropriate, not enterprise)

JWT-based auth with 3–4 hardcoded roles and route-level RBAC ([Chapter 14](#)) is sufficient. No SSO, no ABAC policy engine, no mTLS service mesh — these are enterprise upgrade triggers, not MVP requirements ([Chapter 19](#)).

4.6 Maintainability

- Single Python codebase for the backend (FastAPI + LangGraph + retrieval utilities); single Next.js codebase for the frontend — two repositories, not eighteen services.
- Configuration via environment variables ([Chapter 17](#)), no hardcoded credentials or endpoints.
- Structured logging on every pipeline stage and every agent step, sufficient to debug a demo failure within minutes, not hours.
- Context-aware traversal and compositional reasoning are implemented as pure application-level functions and agent tools inside the existing `fabric-api` service — no new deployable unit is introduced by either capability.

4.7 Portability

Docker Compose on one workstation or one cloud VM is the only deployment target. No Kubernetes manifests, no multi-node orchestration ([Chapter 17](#)).

4.8 Usability

The graph explorer is the primary navigation surface and must be usable by a judge with zero onboarding: search, click, read. The UI must remain acceptable on a phone browser ([Chapter 13](#)) without requiring an installed app.

Chapter 5

System Architecture

5.1 The whole system in one diagram

Before the layer-by-layer breakdown below, this is the single picture that explains CORTEX end to end — from a raw document upload to a cited, on-screen answer. Every chapter after this one is a detailed explanation of exactly one box in this diagram; nothing in this document introduces a stage that does not appear here.

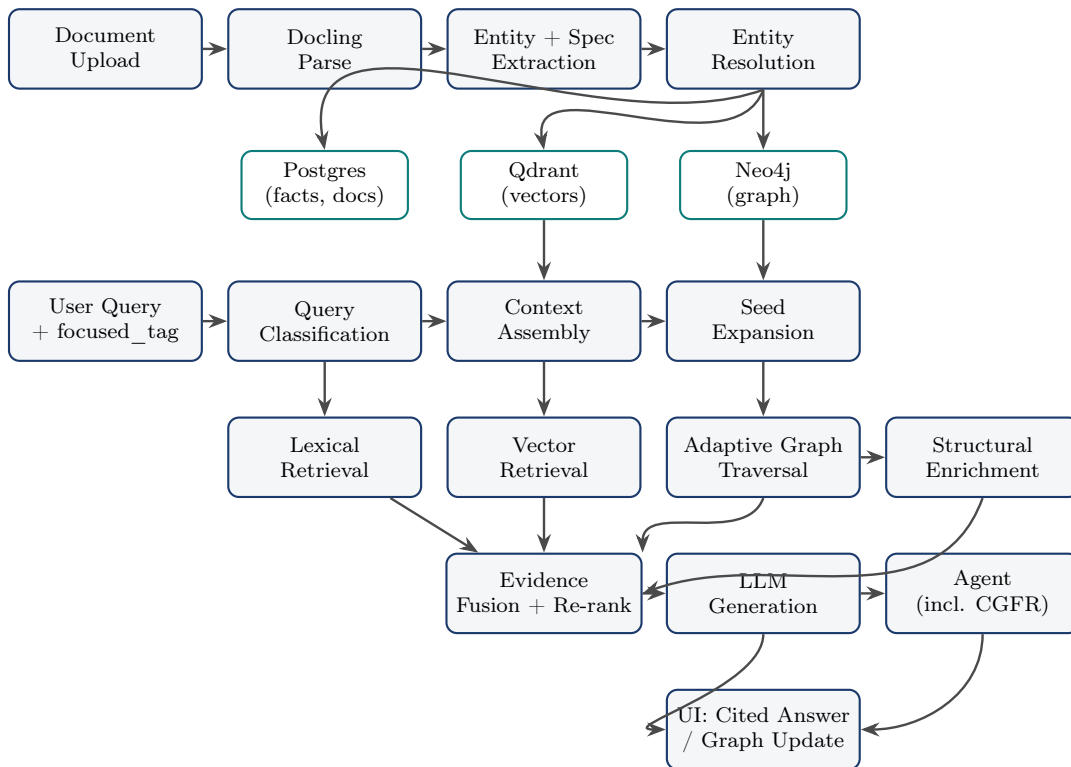


Figure 5.1: Master pipeline: the entire CORTEX system in one diagram. Top: ingestion (Docling parse → extraction → resolution → Neo4j/Qdrant/Postgres). Middle: a query is classified, its context assembled (query + chat history + focused graph node), and fanned out across adaptive graph traversal, vector, and lexical retrieval. Bottom: fused evidence feeds generation and, where relevant, an agent (including compositional CGFR reasoning), both landing back in the UI as a cited answer or a live graph update.

5.2 Key technology decisions at a glance

Five choices carry most of the architectural weight in Figure 5.1. The full rationale and rejected alternatives for every technology choice are in the [Decision Log](#) appendix; this table is the one-line version for a reader who only has thirty seconds per decision.

Choice	Why, in one line
Neo4j (Community)	Mature Cypher tooling and the graph-library ecosystem needed for both fixed-depth and adaptive traversal, without paying for GDS Enterprise at demo scale
Qdrant	Single-binary vector store with native hybrid (dense+sparse) search, reused directly by both the vector retrieval pathway and graph seed expansion — one collection, two jobs
Docling	Per-element bounding-box provenance is what makes passage-level citation possible at all; strongest table-structure recognition of the options considered
LangGraph	Deterministic supervisor-worker orchestration with typed tool contracts and state checkpointing, so a 4-person team can reason about agent control flow under a 5-week clock
One FastAPI app for Fabric + Retrieval + Agents	Removes network hops (and their auth, retries, observability cost) between retrieval, generation, and agent logic — context-aware traversal and CGFR are functions inside this app, not new services

Table 5.1: Key technology decisions at a glance

5.3 Three-layer architecture

The system is exactly three logical layers, deployed as four Docker Compose service groups, matching the top-to-bottom structure of Figure 5.1. Context-aware retrieval and compositional reasoning are both new *capabilities* inside this same shape — neither adds a box or an arrow beyond what Figure 5.1 already shows.

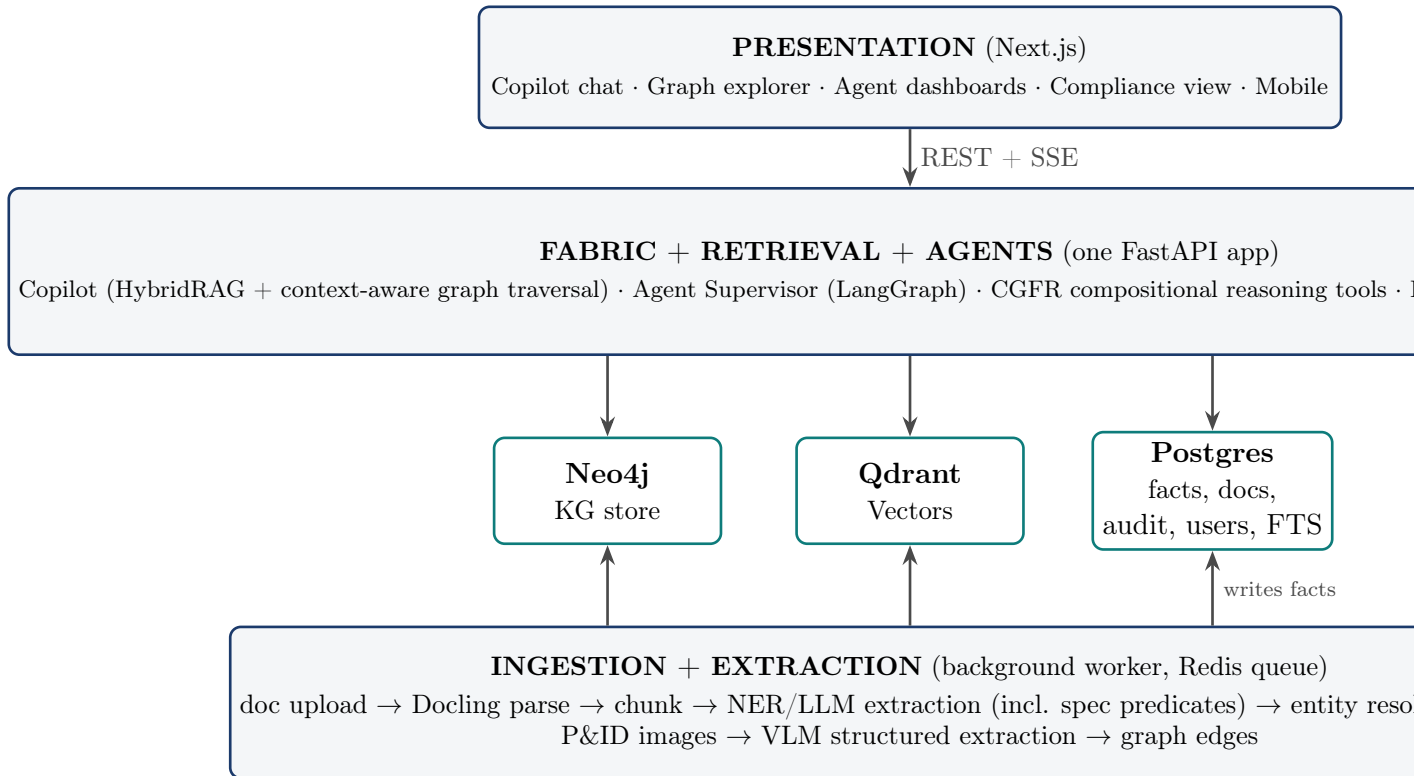


Figure 5.2: Three-layer architecture, deployed as four Docker Compose service groups — unchanged by the context-aware retrieval and compositional reasoning capabilities

5.4 Layer 1: Presentation (Next.js)

Purpose. Render the graph-first UI and stream Copilot / agent output. **Responsibilities.** Authentication UI and token storage; graph rendering via Cytoscape.js; SSE/WebSocket client for streaming answers and agent steps; role-aware UI gating; tracking the currently focused graph node and forwarding it as `focused_tag` on every query (Chapter 9); rendering CORTEX Asset’s compositional comparisons as structured tables (Chapter 13). **Inputs.** User interaction events (search, click, confirm/dismiss); streamed server events. **Outputs.** REST calls to the Fabric layer; rendered graph, chat, and dashboard views. **Dependencies.** Fabric layer’s REST/SSE/WS surface (Chapter 12). Full detail in Chapter 13.

5.5 Layer 2: Fabric + Retrieval + Agents (one FastAPI app)

Purpose. The single backend service that owns all business logic: retrieval, generation, agent orchestration, and graph/entity queries. **Responsibilities.**

- Expose the Copilot endpoint (`/api/query`) implementing HybridRAG with context-aware graph traversal (Chapter 9).
- Host the LangGraph Supervisor and its three workers, including CORTEX Asset’s compositional (CGFR) reasoning mode (Chapter 11).
- Expose entity and graph query endpoints backed by Neo4j.
- Enforce JWT auth and role-based route guards on every endpoint (Chapter 14).
- Write every query and agent action to `audit_log`.

Inputs. REST requests from the frontend; background-queue completion events from the in-

gestion worker (new facts available). **Outputs.** Cited answers (SSE), agent reasoning streams (SSE/WS), graph query results (JSON), audit records. **Dependencies.** Neo4j, Qdrant, Postgres, Redis (for agent job queue if async agent invocation is used), the frontier and fast-tier LLM providers. **Why one app, not many services.** See Section 5.2; a single FastAPI process is precisely why context-aware traversal and compositional reasoning are implemented as functions and agent tools *inside* this app rather than as a new retrieval microservice or a new reasoning microservice.

5.6 Layer 3: Ingestion + Extraction (background worker)

Purpose. Turn raw uploaded documents into facts, embeddings, and graph edges, asynchronously, so the upload endpoint returns immediately. **Responsibilities.** Docling parsing; pandas/openpyxl loading for spreadsheets; chunking; NER/LLM structured extraction against the extended predicate set (including specification-type predicates, Chapter 8); 2.5-stage entity resolution; VLM extraction for P&ID images; writing candidate facts to Postgres, embeddings to Qdrant, and materialising active facts into Neo4j. **Inputs.** Uploaded files (via `/api/documents`), queued as jobs in Redis. **Outputs.** Rows in `facts`, `document_registry`, `entity_registry`; vectors in Qdrant; nodes/edges in Neo4j. **Dependencies.** Redis (queue), Docling, the fast-tier LLM, the embedding model, Postgres, Neo4j, Qdrant. Full detail in Chapter 6.

5.7 Data stores

Store	Role
Neo4j Community	Knowledge graph of active facts, including specification-type relationships; traversal queries (both shallow fixed-depth and adaptive relevance-decayed) for agents and the graph explorer
Qdrant	Chunk-level embeddings for vector similarity retrieval, with doc/page/bbox metadata; reused directly by the seed-expansion phase of context-aware traversal
PostgreSQL	System of record for facts (all statuses, including specification facts), document registry, entity registry/aliases, users, audit log, and full-text search index
Redis	Lightweight async job queue for ingestion (RQ)

Table 5.2: Data stores and their roles

5.8 Request lifecycle: Copilot query

This is Figure 5.1’s middle section, spelled out step by step:

1. **Client sends query + JWT + session context** to `POST /api/query`. The payload carries `query`, `session_id`, and an optional `focused_tag` — the tag of whichever graph node the user currently has selected in the explorer (Chapter 9).
2. **Role check (coarse RBAC).** FastAPI dependency validates the JWT signature and expiry, then checks the requested scope against the role claim; a failing check returns 403 with a machine-readable reason.
3. **Query classification.** A single fast-tier LLM call (or a regex/keyword pre-filter with the LLM as fallback) tags the query as *factual* / *diagnostic* / *procedural* / *open*. This tag drives both the fusion weights (Chapter 9, Table 9.9) and the shallow-vs-full traversal depth decision.

4. **Context assembly and parallel fan-out.** The graph pathway first assembles a rich seed context (current query entities, recent chat-history entities, the focused node) and then three async retrieval calls run concurrently: the context-aware graph traversal ([Chapter 9](#)), Qdrant vector search, and Postgres FTS/BM25 lexical search. Each has its own timeout; a slow pathway is dropped from fusion rather than blocking the other two.
5. **Merge + re-rank.** Candidates from all three pathways — including the synthetic passages produced by graph traversal — are merged, deduplicated by passage ID, and passed through the local cross-encoder to select the top 5–8 chunks.
6. **Generation + citation check.** The frontier model generates a cited answer from the selected chunks; a citation-resolution check parses every `[doc_id:passage_id]` tag and rejects the answer once, regenerating with an explicit instruction to only use resolvable tags, if any tag fails to resolve.
7. **Agent trigger check.** If the query or its classification matches an agent-trigger pattern — including a component-selection or compatibility pattern that routes to CORTEX Asset’s compositional reasoning mode ([Chapter 11](#)) — the Supervisor is invoked asynchronously and streams a follow-up event over the same SSE channel.
8. **Response + audit.** The final answer and its citations are returned to the client; the query, the resolved pathway weights, and the result are written to `audit_log`.

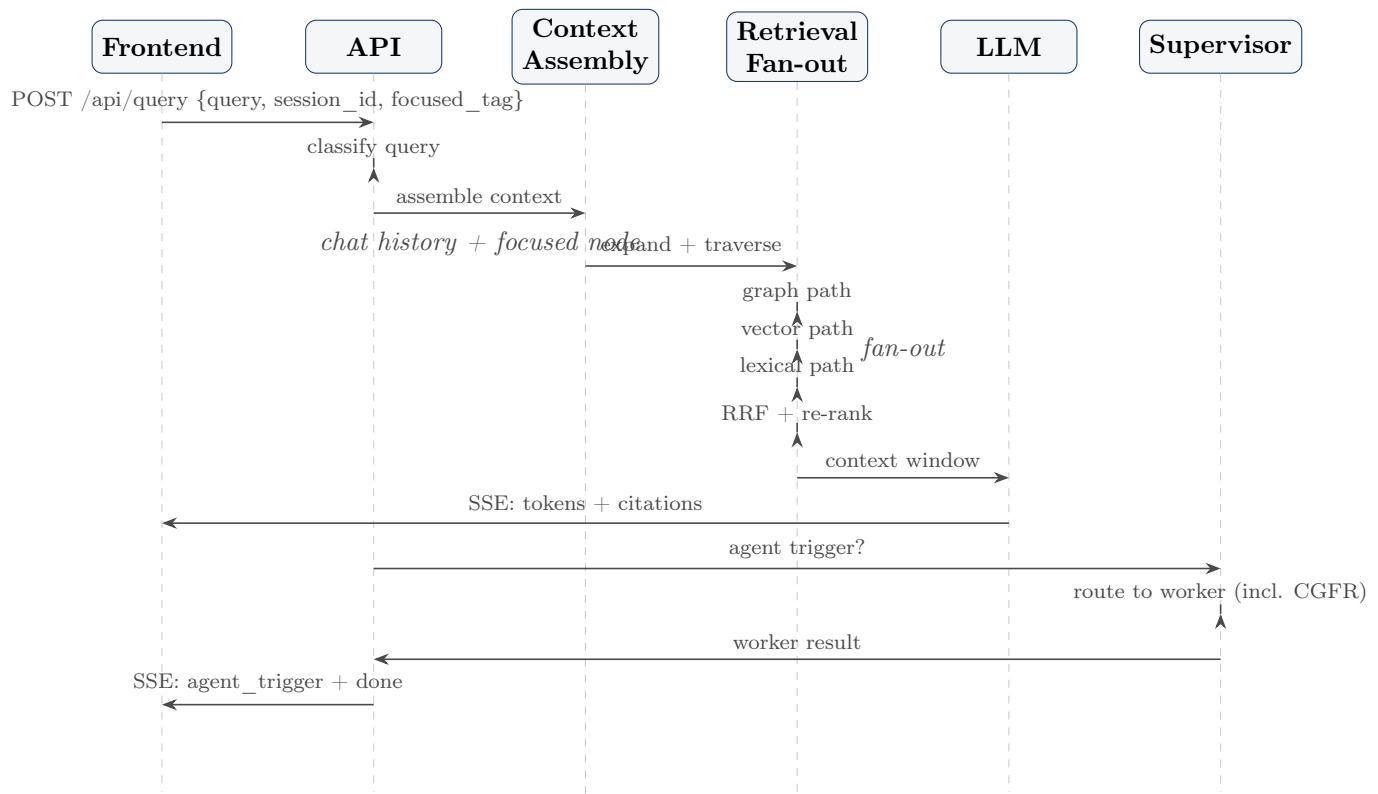


Figure 5.3: Sequence diagram: Copilot query through context assembly, retrieval, generation, and optional agent follow-up (including compositional reasoning) — the same flow as [Figure 5.1](#), shown as a timeline instead of a pipeline

5.9 Component interaction summary

From	To	Interaction
Frontend	Fabric API	REST for CRUD/query, SSE for streaming answers, WS for Copilot session; <code>focused_tag</code> included on every query
Fabric API	Neo4j	Cypher via the official Neo4j Python driver, connection-pooled; both fixed-depth and batched adaptive-traversal queries
Fabric API	Qdrant	gRPC/HTTP via <code>qdrant-client</code> , one collection; reused by both the vector retrieval pathway and the graph seed-expansion phase
Fabric API	Postgres	<code>asyncpg</code> /SQLAlchemy async engine
Fabric API	Redis	Job enqueue for ingestion (RQ), read of job status
Ingestion worker	Postgres/Neo4j/Qdrant	Writes candidate facts (including specification facts), embeddings, and graph edges
Fabric API	LLM providers	Direct SDK calls (fast tier and frontier tier), see Chapter 10

Table 5.3: Component interaction summary

5.10 Demo corpus target

The design point for every latency and capacity decision in this document is ~ 100 – 300 industrial documents (a realistic mix of PDFs, a handful of scanned pages, a couple of spreadsheets including a spare parts catalog with dimensional/material specifications, and 2–3 P&ID drawings), covering roughly 15–25 pieces of equipment with enough overlapping work orders, inspection reports, and OEM manual pages to make the graph genuinely connected — and enough specification data on both equipment and spare parts to make compositional reasoning demonstrable.

Chapter 6

Data Flow

6.1 End-to-end pipeline

The stage list below is the ingestion half of Figure 5.1 (Section 5.1), plus the retrieval/agent/frontend stages that consume what ingestion produces, so each box in the master diagram has exactly one place it is explained in full.

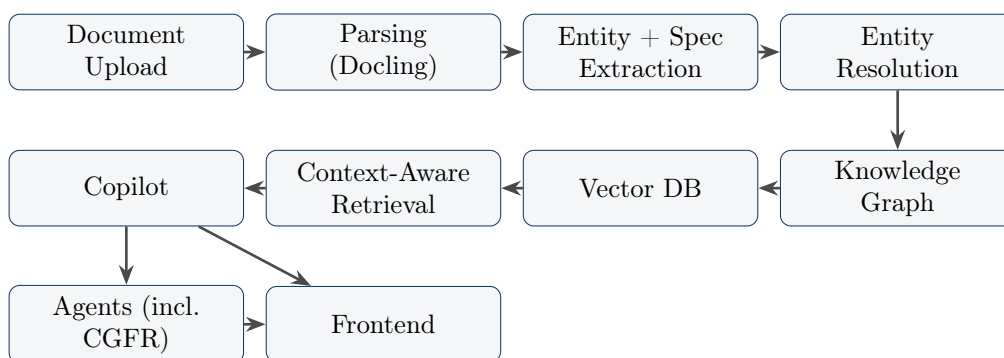


Figure 6.1: End-to-end data flow — the same ten stages as Figure 5.1, laid out linearly

6.2 Stage 1: Document Upload

Purpose. Accept a file, register it, and hand it to the async ingestion queue without blocking the caller. **Inputs.** Multipart file upload (PDF, XLSX, or image) via POST `/api/documents`. **Outputs.** A `document_registry` row with status `queued`; a Redis/RQ job ID. **Algorithm.** (1) Validate MIME type and size; (2) store the raw file to a local/mounted volume keyed by a generated `doc_id` (UUID); (3) insert a `document_registry` row (`doc_id`, `type`, `authority_class`, `ingested_at`, `status='queued'`); (4) enqueue an ingestion job referencing `doc_id`; (5) return `doc_id` and `status_url` to the caller. **Edge cases.** Duplicate filename (allowed — `doc_id` is the identity, not the filename); zero-byte upload (rejected, 422); unsupported MIME type (rejected with a clear message naming supported types). **Error handling.** Storage write failures return 500 and do not enqueue a job for a file that was never durably stored. **Logging.** `doc_id`, filename, size, MIME type, uploader (from JWT), timestamp. **Expected latency.** < 500ms (excludes the async pipeline that follows). **Dependencies.** Postgres, Redis, local/volume storage.

6.3 Stage 2: Parsing (Docling)

Purpose. Convert a raw document into structured text with layout and bounding-box provenance. **Inputs.** Raw file path from Stage 1. **Outputs.** A structured document object: ordered elements (paragraph, heading, table cell, figure) each with page number, bounding box, and element type; routed separately for spreadsheets. **Algorithm.** PDF/DOCX → Docling’s layout+OCR+table pipeline, producing per-element bounding boxes used later for citations. XLSX/CSV → pandas/openpyxl loader producing one structured record per sheet/row, with a synthetic bounding box (null page, cell range as the locator) so the same citation schema applies uniformly — this is how spare-parts-catalog rows (dimensions, material, rated pressure/temperature) become citable facts. **Edge cases.** Scanned pages with poor OCR confidence are still passed through (Docling’s OCR path) with a lower per-element confidence flag propagated to Stage 3 extraction confidence. Password-protected or corrupt files fail this stage explicitly, marking `document_registry.status='parse_failed'` rather than silently skipping. **Error handling.** A parse failure on one document does not block the batch; the job is marked failed and logged, ingestion of the remaining queue continues. **Logging.** Element count, page count, OCR-confidence distribution, parse duration. **Expected latency.** A few seconds to ~30s per document depending on page count and whether OCR is invoked. **Dependencies.** Docling, pandas/openpyxl. **Future extension.** None planned; Docling covers the MVP’s full format set (Chapter 19 adds no new parser).

6.4 Stage 3: Entity & Specification Extraction

Purpose. Turn parsed elements into candidate entities, relational facts, and specification facts. **Inputs.** Structured document elements from Stage 2 (chunked per Chapter 9’s chunking scheme). **Outputs.** Candidate (subject, predicate, object) triples with a source passage reference, an `extraction_method` tag (RuleBased for simple tag-pattern regex hits, LLM for structured extraction calls), and a raw confidence score. **Algorithm.** (1) A rule-based pass matches known tag-number patterns (e.g. `P-\d{3}[A-Z]?`) directly against the ontology’s Equipment type. (2) A fast-tier LLM call performs JSON-schema-constrained structured extraction per chunk, prompted with the *extended* ontology’s entity and relationship types (Chapter 8, including the specification-type predicates below) and instructed to only extract what the passage states, never to infer beyond it. (3) P&ID images bypass text extraction entirely and go through the VLM path (Stage 3b) instead.

```

1  # Relational predicates (unchanged from the original ontology)
2  PREDICATES = [
3      "experiencedFailure", "hasProcedure", "connectedTo",
4      "regulatedBy", "addresses", "usesPart", "mentions",
5  ]
6
7  # Specification-type predicates, required for compositional reasoning
8  # (Ch. 11, Section "Compositional Reasoning") -- prompt-level extension only,
9  # no extraction-service code change and no graph-schema migration.
10 PREDICATES_EXTENDED = [
11     *PREDICATES,
12     "hasSpec",           # Equipment -> spec value (diameter, weight, ...)
13     "hasMaterial",       # Equipment/SparePart -> material grade
14     "operatingTemp",     # Equipment -> operating temperature
15     "operatingPressure", # Equipment -> operating pressure
16     "fitsRange",         # SparePart -> compatible dimension range
17     "compatibleWith",    # Material -> Material or Chemical
18     "ratedFor",          # SparePart -> operating condition limit
19 ]

```

Listing 6.1: Extended predicate set fed to the extraction prompt (Ch. 8’s relational predicates, extended with specification-type predicates)

This is a *prompt-level* extension only: the JSON schema for structured extraction (Chapter 10) simply lists more allowed predicate values, and the Neo4j write path (Chapter 8) already MERGES arbitrary relationship types, so no extraction-service code change and no graph-schema migration are required to support it.

Stage 3b: P&ID extraction. A single multimodal (frontier-tier) call per drawing image, prompted to return a JSON list of {symbol_type, tag_if_legible, approximate_bbox} and a list of {tag_a, tag_b, connection_confidence} adjacency pairs inferred from line proximity. These are written as graph edges with lower confidence than text-derived facts (typically ≤ 0.6 vs. ≥ 0.8 for text-derived). **Edge cases.** A chunk with no extractable entities returns an empty list, not an error. Malformed LLM JSON output triggers one retry with a stricter schema reminder before the chunk is logged as `extraction_failed` and skipped. **Error handling.** Extraction failures are per-chunk, not per-document. **Logging.** Triples extracted per document, extraction method breakdown, JSON-parse failure rate, and specifically the fraction of extracted triples that use a specification-type predicate (a leading indicator for whether compositional reasoning will have anything to work with, Chapter 11 §11.5.4). **Expected latency.** Dominated by the number of chunks $\times$ one fast-tier call each; target < 60 s per average document end-to-end for Stages 2–3 combined. **Dependencies.** Fast-tier LLM (bulk extraction), frontier-tier multimodal LLM (P&ID only).

6.5 Stage 4: Entity Resolution

Purpose. Map every extracted mention to a canonical entity so “P-101A”, “Pump 101A”, and “the feed pump” become one graph node. **Inputs.** Candidate triples from Stage 3; the canonical tag list and alias table built during demo-corpus onboarding. **Algorithm (2.5 stages).**

1. **Exact tag match** against the canonical tag list.
2. **Fuzzy string match** (Levenshtein/Jaro–Winkler, threshold ~ 0.85).
3. **Embedding fallback (the “0.5” stage)** for free-text mentions with no tag structure, reusing the RAG embedding model already deployed.

Outputs. Resolved `subject_tag/object_tag` on each triple; unresolved mentions logged. **Edge cases.** Two distinct canonical tags whose fuzzy-match scores tie: resolve to the higher raw string-similarity score and log the ambiguity. **Error handling.** An unresolved mention still allows the surrounding triple’s other side (if resolved) to be written; a triple with both sides unresolved is dropped with a log entry. **Logging.** Resolution stage that fired per mention, unresolved count per document. **Expected latency.** Sub-second per mention for stages 1–2; embedding fallback bounded by the existing embedding model’s call latency. **Dependencies.** Canonical tag list, alias table, RAG embedding model.

6.6 Stage 5: Knowledge Graph

Purpose. Persist resolved facts as the system of record, and materialise active facts into Neo4j for traversal. **Inputs.** Resolved triples from Stage 4. **Outputs.** Rows in the Postgres `facts` table (Chapter 7); corresponding nodes/edges in Neo4j for `status='active'` facts only, including specification-type edges. **Algorithm.** Each resolved triple is written as one `facts` row with a fresh `fact_id, status='active'`. A Neo4j MERGE (on canonical tag) creates or reuses the subject/object node, then MERGES the relationship edge with the fact’s confidence as an edge property. Full detail

and schema in [Chapter 8](#). **Edge cases.** A new fact that contradicts an existing active fact on the same (subject,predicate) does not overwrite it silently; see the fact/trust model in [Chapter 8](#). **Expected latency.** Sub-100ms per fact write; batched per document.

6.7 Stage 6: Vector DB

Purpose. Make every chunk retrievable by semantic similarity — and, as of the context-aware retrieval design, also usable as the substrate for graph seed expansion ([Chapter 9](#)). **Inputs.** Chunks produced during Stage 2/3 chunking (parent-child, structure-aware, capped at 512 tokens per [Chapter 9](#)). **Outputs.** One Qdrant point per chunk: embedding vector + payload {doc_id, page, bbox, authority_class}. **Expected latency.** Embedding call latency dominates; batched per document.

6.8 Stage 7: Context-Aware Retrieval

Detailed fully in [Chapter 9](#). In the data-flow view, this stage consumes the graph, the vector index, and the lexical index built in Stages 5–6, assembles a rich seed context from the current query, chat history, and the focused graph node, adaptively traverses the graph, and produces a ranked, re-ranked set of 5–8 chunks for a given query. This is CORTEX’s core differentiator versus a conventional retrieval pipeline; [Table 2.2](#) in [Chapter 2](#) summarises exactly what it does that plain vector search cannot.

6.9 Stage 8: Copilot

Consumes the re-ranked chunks from Stage 7 and generates a cited answer, as detailed in [Chapter 9](#) and the request lifecycle in [Section 5.8](#).

6.10 Stage 9: Agents (including Compositional Reasoning)

Consumes the graph (Stage 5) and, where relevant, Copilot-style retrieval (Stage 7) to produce worker-specific outputs (risk scores, causal hypotheses, compliance gaps, and — for CORTEX Asset on component-selection questions — constructed, cited compositional recommendations). Full detail in [Chapter 11](#).

6.11 Stage 10: Frontend

Renders Copilot answers, agent outputs, compositional comparison tables, and the graph itself. Full detail in [Chapter 13](#).

6.12 Pipeline-wide error handling policy

Failure	Policy
Single document parse failure	Mark <code>parse_failed</code> , continue batch, surface in an ingestion status view
Single chunk extraction failure	Retry once with stricter schema, else skip chunk, log
Unresolved entity mention	Log, do not block the rest of the document's facts
LLM call timeout (any tier)	One retry; on second failure, surface a degraded response rather than hang
P&ID VLM call failure during demo	Fall back to pre-loaded graph edges for the curated drawings (Chapter 4)
Structural-enrichment phase (Ch. 9) exceeds its latency budget	Drop the phase, proceed with adaptive-traversal results only
CGFR candidate gathering finds zero candidates	Return an explicitly-flagged, general- knowledge-based answer rather than fabricating a match (Chapter 11 , §11.5.4)

Table 6.1: Pipeline-wide error handling policy

Chapter 7

Database Design

7.1 Overview

PostgreSQL is the system of record for facts, documents, entities, users, and audit history; Neo4j holds a derived, active-facts-only graph view; Qdrant holds chunk embeddings. This chapter covers the Postgres schema in full; Neo4j’s graph model is in [Chapter 8](#); Qdrant’s collection schema is in [Chapter 9](#). No table in this chapter changes shape to support specification facts or compositional reasoning — both are accommodated entirely within the existing `facts.predicate` free-text column and the existing `entity_registry.attributes` JSONB column.

7.2 Entity-relationship overview

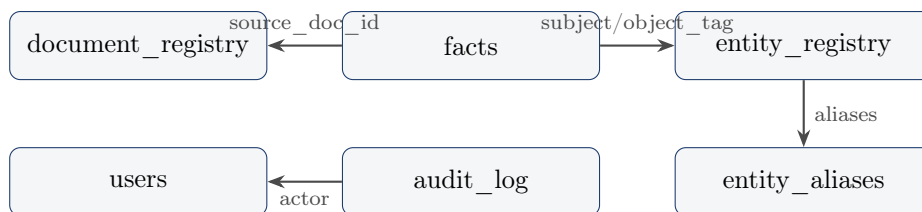


Figure 7.1: Postgres entity-relationship overview

7.3 Table: document_registry

```

1 CREATE TABLE document_registry (
2     doc_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     filename        TEXT NOT NULL,
4     doc_type        TEXT NOT NULL,          -- pdf / xlsx / image
5     authority_class TEXT NOT NULL DEFAULT 'Supporting', -- Authoritative /
6     status          TEXT NOT NULL DEFAULT 'queued',    -- queued / parsing /
7                                     -- parse_failed /
7                                     ingested
8     page_count      INTEGER,
9     uploaded_by     UUID REFERENCES users(id),
10    ingested_at      TIMESTAMPTZ DEFAULT NOW()
11 );
12 CREATE INDEX idx_docreg_status ON document_registry(status);

```

Listing 7.1: document_registry schema

Constraints. `doc_type` and `status` are application-level enums (checked in Pydantic, not a Postgres `CHECK` constraint, to keep migrations trivial during the build). **Indexes.** `status` is indexed for the ingestion dashboard’s status polling.

7.4 Table: facts

This is the trust-model table and the single most important schema in the system.

```

1 CREATE TABLE facts (
2     fact_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     subject_tag      TEXT NOT NULL,
4     predicate        TEXT NOT NULL,          -- free text; see extended predicate
5                                           -- set in Ch. 6 and Ch. 8
6     object_value     TEXT,
7     object_tag       TEXT,
8     source_doc_id    UUID NOT NULL REFERENCES document_registry(doc_id),
9     source_passage   JSONB,                 -- {page, bbox}
10    extraction_method TEXT,                 -- RuleBased / NER / LLM / VLM /
11                                     HumanEntry
12    confidence        NUMERIC(4,3),
13    status            TEXT DEFAULT 'active', -- active / superseded / disputed
14    superseded_by     UUID REFERENCES facts(fact_id),
15    created_at        TIMESTAMPTZ DEFAULT NOW()
16 );
17 CREATE INDEX idx_facts_subject ON facts(subject_tag);
18 CREATE INDEX idx_facts_status ON facts(status);
19 CREATE INDEX idx_facts_doc    ON facts(source_doc_id);
20 CREATE INDEX idx_facts_predicate ON facts(predicate);

```

Listing 7.2: facts schema

No schema change for specification facts. Because `predicate` is a free-text column, specification-type facts (`hasSpec`, `hasMaterial`, `operatingTemp`, `operatingPressure`, `fitsRange`, `compatibleWith`, `ratedFor`) are written to exactly this table, exactly this way, with `object_value` holding the spec’s literal value (e.g. "4in", "316SS", "150C"). `idx_facts_predicate` is the one addition (Week N, trivial migration) that keeps specification-fact lookups by predicate fast at demo scale.

Implementation notes. Only rows with `status='active'` are ever projected into Neo4j (Chapter 8). A correction workflow (`POST /api/entities/{tag}/feedback`) inserts a new `fact_id` with `extraction_method='HumanEntry'` and sets the prior row’s `status='superseded'` and `superseded_by` to the new `fact_id` — an `UPDATE` plus an `INSERT` inside a single transaction, never a `DELETE`. **Edge cases.** A fact whose `object_tag` does not resolve (Stage 4 of Chapter 6) still gets a row, with `object_tag` left `NULL` and `object_value` holding the raw text.

7.5 Table: entity_registry and entity_aliases

```

1 CREATE TABLE entity_registry (
2     tag          TEXT PRIMARY KEY,
3     entity_type  TEXT NOT NULL,  -- Equipment / FailureMode / MaintenanceAction
4     / ...
5     display_name TEXT,
6     attributes  JSONB           -- type, location, criticality, dimensional
7                                     specs,
6                                     -- material grade, operating limits, etc.
7 );

```

```

8
9 CREATE TABLE entity_aliases (
10     alias_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
11     tag         TEXT NOT NULL REFERENCES entity_registry(tag),
12     alias_text  TEXT NOT NULL,
13     source      TEXT          -- how the alias was learned: seed / fuzzy /
                             embedding
14 );
15 CREATE INDEX idx_alias_text ON entity_aliases(alias_text);

```

Listing 7.3: entity_registry and entity_aliases schema

Used by. Entity Resolution Stage 4 (Chapter 6) for exact and fuzzy matching, and by CORTEX Asset’s find_by_type tool (Chapter 11) for pulling candidate components of a given type/subtype (e.g. all SparePart rows with attributes.type = 'funnel'). The attributes JSONB column was already a catch-all before compositional reasoning was designed in; no migration was required to store dimensional specs, material grades, or operating limits there — only a documentation update to this section and a convention (below) for how agents read it.

```

1 {
2   "type": "funnel",
3   "fitsRange": "3in-6in",
4   "hasMaterial": "SS316",
5   "maxTemp": "200C",
6   "maxPressure": "15bar"
7 }

```

Listing 7.4: Convention for specification attributes inside entity_registry.attributes (documentation only, no schema change)

7.6 Table: users

```

1 CREATE TABLE users (
2     id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     username      TEXT UNIQUE NOT NULL,
4     password_hash TEXT NOT NULL,
5     role          TEXT NOT NULL    -- technician / engineer / compliance_officer /
                             admin
6 );

```

Listing 7.5: users schema

Seeded with 3–4 demo accounts per role during Week 1 setup (Chapter 17). Full auth flow in Chapter 14.

7.7 Table: audit_log

```

1 CREATE TABLE audit_log (
2     log_id        BIGSERIAL PRIMARY KEY,
3     actor         UUID REFERENCES users(id),
4     action        TEXT NOT NULL,    -- e.g. 'query', 'agent_invoke', '
                             feedback_submit',
                                     -- 'agent_invoke_cgfr'
5     payload       JSONB,
6     created_at    TIMESTAMPTZ DEFAULT NOW()
7 );
8
9 CREATE INDEX idx_audit_actor ON audit_log(actor);
10 CREATE INDEX idx_audit_created ON audit_log(created_at);

```

Listing 7.6: audit_log schema

Written on every Copilot query, every agent invocation (including CGFR compositional-reasoning invocations of CORTEX Asset), and every feedback submission (Chapter 3, FR-AUTH-2). No DB-trigger-enforced immutability in the MVP (Chapter 19).

7.8 Lexical search

Postgres full-text search (`tsvector`) is used in place of a separate lexical search engine.

```
1 ALTER TABLE facts ADD COLUMN fts tsvector
2   GENERATED ALWAYS AS (to_tsvector('english', coalesce(object_value, '')))
3   STORED;
3 CREATE INDEX idx_facts_fts ON facts USING GIN(fts);
```

Listing 7.7: FTS column and index on facts/document text

Chunk-level text (for passage lexical search) is stored in a `chunks` table mirroring the Qdrant payload, with its own `tsvector` column and GIN index.

7.9 Migrations

Given the 5-week timeline, schema migrations are managed with a single incrementally-applied SQL script per week (`migrations/00N_description.sql`), applied manually via `psql` during Docker Compose startup. The only migration introduced by this document beyond the original schema is `idx_facts_predicate` (above); everything else needed for specification facts and compositional reasoning is a documentation/prompt-level change, not a database change.

Chapter 8

Knowledge Graph

8.1 Purpose and responsibilities

Purpose. Provide a traversable, relational view of active facts, so agents and the graph explorer can answer “what is connected to X” questions that passage retrieval cannot, and so that adaptive, context-aware traversal (Chapter 9) and compositional reasoning (Chapter 11, §11.5.4) have a rich enough substrate — including specification-level facts — to work from. **Responsibilities.** Maintain node/edge consistency with the **facts** table; serve both fixed-depth and adaptive relevance-decayed traversal queries within the latency budget (Chapter 4); expose a Cypher query surface to the Fabric API. **Inputs.** Active facts from Postgres (Chapter 7). **Outputs.** Graph query results (paths, subgraphs) to the Fabric API and, through it, to the graph explorer and agents.

8.2 Ontology

Entity	Key attributes	Example
Equipment	tag, type, location, criticality, and (new) dimensional/material/operating attributes when known	Pump P-101A
FailureMode	name, description	Bearing failure
MaintenanceAction	type, date, description	Bearing replaced
Procedure	doc_number, revision	SOP-MAINT-042
Regulation	code, requirement text	OISD-154
Document	type, date, authority_class {Authoritative, Supporting}	OEM Manual Rev. 3
Personnel	role (name shown only to authorised roles)	Technician
SparePart	part number, spec, and (new) fit-range/material/rating attributes when known	SKF 6310 bearing, Funnel F-42

Table 8.1: Core entity types

Relationship	Direction	Notes
experiencedFailure	Equipment → FailureMode	From maintenance/failure records
hasProcedure	Equipment → Procedure	Links equipment to its SOPs
connectedTo	Equipment → Equipment	From P&ID extraction; lower confidence edges
regulatedBy	Equipment → Regulation	Used by CORTEX Comply
addresses	MaintenanceAction → Equipment	Work order to equipment link
usesPart	MaintenanceAction → SparePart	Parts consumption
mentions	Document → entity	Provenance edge, any entity type
<i>Specification-type relationships (integrated for compositional reasoning, §8.2.1):</i>		
hasSpec	Equipment → spec value	Dimensional or physical spec (diameter, weight)
hasMaterial	Equipment/SparePart → Material	Material grade
operatingTemp	Equipment → temperature value	Operating temperature
operatingPressure	Equipment → pressure value	Operating pressure
fitsRange	SparePart → dimension range	Compatible dimension range for a component
compatibleWith	Material → Material/Chemical	Chemical/galvanic compatibility
ratedFor	SparePart → condition limit	Operating condition rating (max temp, max pressure)

Table 8.2: Core and extended relationships

8.2.1 Extended ontology for compositional reasoning

The relational ontology above (rows 1–7) is exactly what the system needed to answer “what’s connected to this equipment” and “what regulation applies to it.” It is not, on its own, enough to answer “which spare part is compatible with this equipment,” because a compatible spare part and a piece of equipment are frequently never mentioned together in any single document — there is no edge to traverse between them. The seven specification-type predicates in the second half of the table above close this gap: they let CORTEX record what a piece of equipment *needs* (dimension, material, temperature, pressure) and what a spare part *offers* (fit range, material, rating) as separately extracted, separately cited facts, so that an agent can *construct* the missing link at query time by comparing the two sets rather than requiring the link to already exist in the graph (Chapter 11, §11.5.4).

Design decision

Extending the predicate set is a prompt-level change to the extraction LLM (Chapter 10) and a documentation-level convention for `entity_registry.attributes` (Chapter 7). It requires no Neo4j schema migration (Neo4j is schema-free on relationship types via `MERGE`) and no new entity type — `Specification` values are stored as scalar properties on existing `Equipment` and `SparePart` nodes, not as a new node label.

8.3 Neo4j node and edge model

```

1 // Node upsert (subject side), keyed by canonical tag
2 MERGE (e:Equipment {tag: $subject_tag})
3 ON CREATE SET e.type = $equipment_type, e.location = $location,
4               e.criticality = $criticality
5
```

```

6 // Relational edge upsert with confidence and provenance as edge properties
7 MATCH (s {tag: $subject_tag}), (o {tag: $object_tag})
8 MERGE (s)-[r:EXPERIENCED_FAILURE]->(o)
9 ON CREATE SET r.fact_id = $fact_id,
10               r.confidence = $confidence,
11               r.source_doc_id = $source_doc_id,
12               r.created_at = datetime()
13
14 // Specification-type edge upsert -- identical MERGE pattern, new relationship
   type
15 MATCH (e:Equipment {tag: $subject_tag})
16 MERGE (e)-[r:HAS_SPEC {property: 'diameter'}]->(v:SpecValue {value: $spec_value})
17 ON CREATE SET r.fact_id = $fact_id, r.confidence = $confidence,
18               r.source_doc_id = $source_doc_id

```

Listing 8.1: Representative node and edge creation on fact projection, including a specification-type edge

Node labels. One label per ontology entity type, each keyed by its canonical tag: Equipment, FailureMode, MaintenanceAction, Procedure, Regulation, Document, Personnel, SparePart. Specification values are stored either as scalar node properties (e.g. `e.diameter = "4in"`) or, where multiple values of the same property type could exist, as lightweight `SpecValue` nodes reached via the specification-type edges above — this is purely an implementation choice made per-property during Week 2 and does not affect the API surface described in [Chapter 11](#).

Edge types. EXPERIENCED_FAILURE, HAS_PROCEDURE, CONNECTED_TO, REGULATED_BY, ADDRESSES, USES_PART, MENTIONS, and the specification-type edges HAS_SPEC, HAS_MATERIAL, OPERATING_TEMP, OPERATING_PRESSURE, FITS_RANGE, COMPATIBLE_WITH, RATED_FOR. Each edge carries `fact_id`, `confidence`, and `source_doc_id` as properties, so any traversal result — relational or specification — traces to the owning Postgres fact row and source passage.

8.4 Graph materialisation (write path)

Trigger. Every time a fact’s status transitions to `active` (new fact, or a correction accepted), the Fabric API (or the ingestion worker, for bulk loads) issues the corresponding `MERGE` against Neo4j — the same write path handles relational and specification-type facts identically. **Supersession.** When a fact is superseded, its corresponding edge is deleted from Neo4j; the superseded fact remains queryable in Postgres directly.

```

1 MATCH (s {tag: $subject_tag})-[r {fact_id: $old_fact_id}]->(o)
2 DELETE r

```

Listing 8.2: Removing a superseded fact’s edge

8.5 Graph traversal (read path)

8.5.1 Fixed-depth traversal (shallow mode)

```

1 MATCH path = (start:Equipment {tag: $tag})-[:CONNECTED_TO*1..2]-(neighbor:
   Equipment)
2 RETURN path
3 LIMIT 50

```

Listing 8.3: k-hop upstream/downstream traversal, depth ≤ 2 – retained as the fast/shallow path, [Chapter 9 §9.14](#)

```

1 MATCH (e:Equipment {tag: $tag})-[r:EXPERIENCED_FAILURE]->(f:FailureMode)
2 OPTIONAL MATCH (e)-[:ADDRESSES]-(m:MaintenanceAction)-[:USES_PART]->(p:SparePart
3 )
4 RETURN f.name AS failure_mode, count(*) AS frequency, collect(DISTINCT p.
    part_number) AS parts
5 ORDER BY frequency DESC

```

Listing 8.4: Failure-history query used by CORTEX Asset

```

1 MATCH (e:Equipment)-[:REGULATED_BY]->(reg:Regulation {code: $reg_code})
2 WHERE NOT (e)-[:HAS_PROCEDURE]->(:Procedure)-[:MENTIONS]->(reg)
3 RETURN e.tag AS equipment, reg.code AS missing_regulation

```

Listing 8.5: Compliance gap query used by CORTEX Comply

8.5.2 Adaptive, context-aware traversal (full mode)

For diagnostic and open queries, and for agent invocations that ask for “deep” context, the graph is explored via the five-phase, best-first, relevance-decayed pipeline described fully in [Chapter 9](#). That pipeline is the graph’s *primary* read path for anything beyond a simple factual lookup; the fixed-depth query above is the deliberately-retained fast path, not the default. The full pipeline crosses all relationship types — including the specification-type edges introduced in this chapter — so that, for example, a shared **Regulation** node or a shared **SparePart** node can surface as a cross-domain bridge even when it was never directly asked about.

```

1 UNWIND $seed_tags AS seed
2 MATCH path = (start {tag: seed})-[*1..5]-(neighbor)
3 WITH neighbor, min(length(path)) AS min_depth,
4     collect(DISTINCT type(relationships(path)[-1])) AS rel_types,
5     avg([r IN relationships(path) | r.confidence]) AS avg_confidence
6 RETURN DISTINCT neighbor.tag AS tag, labels(neighbor)[0] AS entity_type,
7     min_depth, rel_types, avg_confidence
8 ORDER BY min_depth, avg_confidence DESC LIMIT 200

```

Listing 8.6: Batched multi-hop neighborhood fetch used by adaptive traversal (Ch. 10) – crosses every relationship type, not just **CONNECTED_TO**

8.6 Confidence model on the graph

Every edge inherits the confidence of the fact that produced it. Text-derived facts (extraction via rule-based, NER, or LLM methods, including specification facts pulled from spec tables and catalogs) typically carry confidence ≥ 0.8 ; P&ID-derived **CONNECTED_TO** edges (VLM extraction) are written at ≤ 0.6 . The graph explorer visually distinguishes edges below a 0.7 threshold (dashed, per [Chapter 13](#)) so users can see which connections are inferred versus text-confirmed. During adaptive traversal, edge confidence directly modulates how far a relevance score propagates down a branch ([Chapter 9](#)), so low-confidence P&ID edges naturally receive lower traversal priority without any special-case logic.

8.7 Graph updates and refresh

Neo4j is a *materialised view*, not the system of record — it can be safely rebuilt from Postgres at any time by replaying all active facts.

```

1 def rebuild_graph():
2     for fact in db.query("SELECT * FROM facts WHERE status = 'active'"):

```

```
3 neo4j_session.run(MERGE_QUERY_FOR(fact.predicate), **fact.as_params())
```

Listing 8.7: Full graph rebuild from Postgres (recovery procedure) – identical for relational and specification facts

8.8 Edge cases

- A fact whose `object_tag` is `NULL` (unresolved object) is not projected as an edge; it remains a Postgres-only fact until/unless a later correction resolves it.
- Two facts asserting the same (`subject`, `predicate`, `object`) from different documents both exist in Postgres (different `fact_ids`) but collapse to one `MERGED` edge in Neo4j, with the edge’s `fact_id` property pointing at whichever fact was written last.
- A specification fact with no matching candidate on the other side (e.g. equipment specs exist but no compatible spare part was ever extracted) is not an error condition; CORTEX Asset’s compositional reasoning explicitly reports this rather than fabricating a match (Chapter 11, §11.5.4).

8.9 Testing

Unit tests assert that a fact insertion produces exactly the expected Cypher `MERGE` call (mocked driver), for both relational and specification-type predicates; integration tests run the actual traversal queries above — both fixed-depth and adaptive — against a seeded demo-scale Neo4j instance and assert expected node/edge counts. See Chapter 16.

8.10 Expected latency

Fixed-depth (shallow) traversal targets < 300ms P95; adaptive full-pipeline traversal targets < 500ms P95 total across all five phases (Chapter 9); write-path `MERGE` calls target sub-100ms per fact.

8.11 Dependencies

Neo4j Community (Docker), the official Neo4j Python driver, Postgres (as the upstream source of facts).

8.12 Future extension

Ontology versioning, a dedicated `Specification` node label with typed properties, ClickHouse/DuckDB analytical mirrors, and Memgraph are named upgrade paths in Chapter 19, triggered only at data volumes far beyond the demo corpus.

Chapter 9

Retrieval

9.1 Purpose and responsibilities

Purpose. Given a natural-language query, return the 5–8 passages most likely to support a correct, citable answer, by combining three complementary retrieval pathways — graph, vector, lexical — rather than relying on vector similarity alone, and by exploring the graph *adaptively*, from a context that spans more than the current query string. This chapter is the detailed implementation of the retrieval half of Figure 5.1 (Section 5.1) and of the “Contextual Retrieval” pillar in Chapter 2. **Responsibilities.** Query classification; context assembly; adaptive graph traversal; parallel pathway execution; fusion; re-ranking; chunking policy; citation-resolution validation.

What changed, and why it lives here

Earlier exploratory work considered a rigid Neo4j `CONNECTED_TO *1..2` traversal seeded only by entities named in the current query. That approach produced three concrete failure modes documented in Table 9.1 below. The fix is not a new retrieval system; it is a richer implementation of the same `graph_pathway()` function that already feeds the same fusion step. Nothing about the request lifecycle (Section 5.8), the fusion weights, or the re-ranker changes — only how the graph pathway itself decides what to explore.

9.1.1 Why a fixed hop count was not enough

Failure mode	Example	Why a fixed hop count misses it
Disconnected relevance	User asks about P-101A; Pump P-201B (a different system) had an identical bearing failure last year, structurally 5+ hops away	Fixed depth=2 never reaches it
Implicit context	User has been discussing cooling systems for 4 messages, then asks “what about the valve?” with no entity named	Zero seed entities extracted → the pathway returns empty
Cross-domain bridges	A regulation (OISD-154) connects two equipment clusters through <code>regulatedBy</code> edges	A <code>CONNECTED_TO</code> -only traversal ignores every other relationship type

Table 9.1: Failure modes of a fixed-depth, query-only graph traversal

9.2 Chunking

Policy. Structure-aware, parent-child chunking derived directly from Docling’s element boundaries (headings, paragraphs, table rows), capped at 512 tokens per child chunk, with the parent (e.g. the enclosing section) retrievable for additional context when a child chunk is selected. **Output schema (Qdrant payload).**

```

1 {
2   "chunk_id": "uuid",
3   "doc_id": "uuid",
4   "parent_chunk_id": "uuid|null",
5   "page": 12,
6   "bbox": [x0, y0, x1, y1],
7   "authority_class": "Authoritative",
8   "text": "...
9 }
```

Listing 9.1: Chunk payload schema

9.3 Query classification

Purpose. Tag each query so pathway weights, and the traversal depth mode below, can be set without a trained classifier. **Algorithm.** A keyword/regex pre-filter checks for diagnostic markers (“why”, “keeps failing”, “root cause”), procedural markers (“how do I”, “steps to”), factual markers (“what is”, “when was”), and compositional/component-selection markers (“which X should I use for”, “is X compatible with”, “what fits”); ambiguous queries fall through to a single cheap LLM classification call returning one of {factual, diagnostic, procedural, open}. A compositional marker does not introduce a fifth query-type value; it is used purely as an agent-routing signal (Chapter 11) — retrieval-side, a compositional question is treated as *open*, since it benefits from the same broad, structurally-enriched context. **Why not a learned classifier.** No labelled training data and no time to build one exist for this timeline; the rule-based router with an LLM fallback gives the same architectural story — adaptive fusion weights — with zero ML-training dependency.

9.4 Context Assembly

Goal. Build a rich seed set from more than just the current query string, so that the implicit-context failure mode in Table 9.1 cannot occur.

Source	What it provides	Cost
Current query	Explicit entity mentions (via existing entity resolution, Chapter 6)	Already exists
Chat history (last 5 messages)	Previously discussed entities and their resolved tags	New: parse <code>session_id</code> → last 5 messages
Active document context	If the user clicked a node in the graph explorer, that node’s tag	New: frontend sends <code>focused_tag</code> in the query payload
Query classification	The <code>factual/diagnostic/procedural/open</code> tag	Already exists

Table 9.2: Inputs to context assembly

```

1 @dataclass
2 class TraversalContext:
3     explicit_tags: list[str]          # from current query entity resolution
4     implicit_tags: list[str]          # from chat history + focused node
5     query_type: QueryType             # from classify_query()
6     query_embedding: list[float]      # for relevance scoring during traversal
7
8 async def assemble_context(
9     query: str, session_id: str, focused_tag: str | None
10 ) -> TraversalContext:
11     explicit = resolve_entities(query)
12
13     history = await get_recent_messages(session_id, limit=5)
14     history_tags = set()
15     for msg in history:
16         history_tags.update(resolve_entities(msg.text))
17
18     implicit = list(history_tags - set(explicit))
19     if focused_tag and focused_tag not in explicit:
20         implicit.insert(0, focused_tag)
21
22     return TraversalContext(
23         explicit_tags=explicit,
24         implicit_tags=implicit[:5],    # cap to prevent context explosion
25         query_type=await classify_query(query),
26         query_embedding=await embed(query),
27     )

```

Listing 9.2: Context assembly (Phase 1 of the graph pathway)

API change. The existing POST `/api/query` payload gains one optional field, sent automatically by the frontend whenever a node is selected in the graph explorer ([Chapter 13](#)):

```

1 { "query": "Why does P-101A keep failing?",
2   - "session_id": "s-88a2" }
3 + "session_id": "s-88a2",
4 + "focused_tag": "P-101A" }

```

Listing 9.3: Query payload, before and after

9.5 Seed Expansion

Goal. Turn the assembled context into a broader, ranked set of seed nodes, without relying solely on direct entity mentions.

Strategy A — Direct Seeds. Every tag in `explicit_tags` and `implicit_tags` becomes a seed node.

Strategy B — Embedding-Nearest Nodes. Reuse the vector pathway’s own Qdrant search: query for the top-10 nearest chunks, extract `subject_tag/object_tag` from the `facts` table for those chunks’ source documents, and add those tags as expansion seeds. This catches semantically relevant nodes the user didn’t name, at essentially no additional cost since the same Qdrant search is also run for the vector pathway.

```

1 async def embedding_expand(query_embedding: list[float]) -> list[str]:
2     chunks = await qdrant_search(query_embedding, top_k=10)
3     doc_ids = [c.payload["doc_id"] for c in chunks]
4     rows = await db.fetch("""
5         SELECT DISTINCT subject_tag FROM facts
6         WHERE source_doc_id = ANY($1) AND status = 'active' LIMIT 10

```

```

7     """", doc_ids)
8     return [r["subject_tag"] for r in rows]

```

Listing 9.4: Embedding-based seed expansion (Phase 2, Strategy B) – reuses the vector pathway’s Qdrant search

Strategy C — Type-Based Expansion. For diagnostic queries, also pull equipment of the same type as the primary seed, to find analogous failure patterns:

```

1 MATCH (seed:Equipment {tag: $tag})
2 MATCH (sibling:Equipment)
3 WHERE sibling.type = seed.type AND sibling.tag <> seed.tag
4 RETURN sibling.tag LIMIT 5

```

Listing 9.5: Type-based seed expansion (Phase 2, Strategy C)

Ranking. All expanded seeds are scored:

$$\text{score}(\text{seed}) = w_{\text{explicit}} \cdot \text{is_explicit} + w_{\text{embedding}} \cdot \text{embedding_similarity} + w_{\text{type}} \cdot \text{type_match} + w_{\text{recency}} \cdot \text{recency_in_c}$$

Weights vary by query type (diagnostic queries weight w_{type} higher; factual queries weight w_{explicit} higher). Only the top **15 seeds** proceed to adaptive traversal.

9.6 Adaptive Traversal

Goal. Explore the graph outward from seeds with dynamic depth, stopping when information becomes irrelevant rather than at a hardcoded hop limit — this is what closes both the disconnected-relevance and the cross-domain-bridge failure modes in Table 9.1.

Instead of *1..2, the pathway uses a **best-first traversal with a decaying relevance score**. Traversal along a branch stops when the score drops below a threshold; the whole traversal is bounded by a hard node budget so latency stays predictable regardless of graph density.

```

1 async def adaptive_traverse(
2     seeds: list[RankedSeed], query_embedding: list[float],
3     max_depth: int = 5, relevance_threshold: float = 0.3, max_nodes: int = 50,
4 ) -> list[ScoredNode]:
5     visited, frontier = {}, [(s.tag, 0, s.score) for s in seeds]
6
7     while frontier and len(visited) < max_nodes:
8         frontier.sort(key=lambda x: -x[2]) # best-first
9         tag, depth, parent_score = frontier.pop(0)
10        if tag in visited or depth > max_depth:
11            continue
12
13        node_text = await get_node_text(tag) # cached
14        semantic_sim = cosine_similarity(query_embedding, await embed(node_text))
15        node_score = 0.5 * parent_score * 0.7 + 0.5 * semantic_sim #
16        # DECAY_FACTOR=0.7
17
18        if node_score < relevance_threshold and depth > 1:
19            continue # prune this branch
20        visited[tag] = ScoredNode(tag=tag, score=node_score, depth=depth)
21
22        for neighbor_tag, rel_type, confidence in await get_neighbors(tag):
23            if neighbor_tag not in visited:
24                # edge confidence modulates the propagated score
25                frontier.append((neighbor_tag, depth + 1, node_score * confidence))

```

```
26 return sorted(visited.values(), key=lambda n: -n.score)
```

Listing 9.6: Adaptive, relevance-decayed traversal (Phase 3)

Design decision	Rationale
Best-first, not breadth-first	Explores the most promising branches first within the node budget
All relationship types	Crosses <code>CONNECTED_TO</code> , <code>experiencedFailure</code> , <code>regulatedBy</code> , <code>hasProcedure</code> , and every specification-type edge (Chapter 8) — solving the cross-domain bridge problem
Edge confidence as propagation weight	P&ID-derived edges (confidence ≤ 0.6) naturally get less traversal priority than text-confirmed edges (≥ 0.8), with no special-case logic
Hard cap of 50 nodes	Guarantees latency stays bounded regardless of graph density
<code>DECAY_FACTOR = 0.7</code>	At depth 3, parent contribution is $0.7^3 \approx 0.34$ of original — past this, a node must have high semantic similarity to survive

Table 9.3: Key design decisions in adaptive traversal

Batched Cypher implementation. Rather than running N individual Cypher queries, the multi-hop neighbourhood for all seeds is fetched in one call and scored in Python (query shown in Chapter 8); this keeps Cypher execution to a single round trip ($\leq 100\text{ms}$ at demo scale) and moves the intelligence to application code, which is easier to tune and test under a 5-week clock than equivalent logic embedded in Cypher.

9.7 Structural Enrichment

Goal. Discover macro-patterns that per-node traversal misses — shortest paths between seed clusters, and lightweight community detection — and only pay for this when the query type actually benefits from it.

Multi-seed shortest paths. When the context has multiple seeds from different parts of the graph, the connecting path between them often contains the most relevant intermediate nodes:

```
1 MATCH path = shortestPath(
2     (a {tag: $tag_a})-[*..6]-(b {tag: $tag_b})
3 )
4 RETURN [n IN nodes(path) | n.tag] AS path_tags,
5         length(path) AS path_length,
6         [r IN relationships(path) | type(r)] AS rel_types
```

Listing 9.7: Shortest path between a pair of explicit seeds

```
1 async def find_bridge_nodes(explicit_tags: list[str], already_found: set[str]) ->
2     list[str]:
3     bridges = Counter()
4     for tag_a, tag_b in combinations(explicit_tags, 2):
5         path = await neo4j_shortest_path(tag_a, tag_b, max_depth=6)
6         if path:
7             for tag in path[1:-1]: # exclude endpoints
8                 bridges[tag] += 1
```

```

8     return [t for t, _ in bridges.most_common(10) if t not in already_found]

```

Listing 9.8: Bridge-node discovery (Phase 4a)

Lightweight hub detection. Rather than a full Neo4j GDS Enterprise license (explicitly excluded, [Chapter 19](#)), the MVP approximates community structure with a degree-based heuristic: nodes with high local connectivity in a seed’s neighbourhood are likely community hubs worth including.

```

1 MATCH (seed {tag: $tag})-[*1..3]-(member)
2 WITH member, count(*) AS connections
3 WHERE connections >= 3
4 RETURN member.tag, labels(member)[0], connections
5 ORDER BY connections DESC LIMIT 5

```

Listing 9.9: Degree-based hub detection (Phase 4b), no GDS license required

Query type	Shortest paths?	Hub detection?	Rationale
Factual	No	No	Usually answered by direct neighbours
Diagnostic	Yes	Yes	Root causes often span systems
Procedural	No	Yes (light)	Procedures often reference hub equipment
Open	Yes	Yes	Exploratory queries benefit most from broad context

Table 9.4: When structural enrichment runs

9.8 Passage Synthesis

Scored nodes and paths are converted into synthetic passages and injected into the same fusion pipeline used by vector and lexical retrieval:

```

1 def node_to_passage(node: ScoredNode, edges: list[Edge]) -> SyntheticPassage:
2     lines = [f"{node.tag} ({node.entity_type}):"]
3     for edge in edges:
4         lines.append(
5             f"    - {edge.rel_type} -> {edge.target_tag} "
6             f"[confidence: {edge.confidence:.2f}, source: {edge.source_doc_id}]"
7         )
8     return SyntheticPassage(
9         chunk_id=f"graph-{node.tag}", text="\n".join(lines), score=node.score,
10        source="graph_traversal", fact_ids=[e.fact_id for e in edges],
11    )

```

Listing 9.10: Node-to-passage conversion (Phase 5)

```

1 async def graph_pathway(
2     query: str, query_type: QueryType, session_id: str, focused_tag: str | None,
3 ) -> list[Chunk]:
4     ctx = await assemble_context(query, session_id, focused_tag)
5     seeds = await expand_seeds(ctx)
6     traversed = await adaptive_traverse(seeds, ctx.query_embedding)
7
8     if query_type in (QueryType.DIAGNOSTIC, QueryType.OPEN):
9         bridges = await find_bridge_nodes(ctx.explicit_tags, {n.tag for n in
10            traversed})
11         hubs = await detect_hubs(ctx.explicit_tags)

```

```

11     traversed.extend(bridges + hubs)
12
13     passages = []
14     for node in traversed[:20]:
15         edges = await get_node_edges(node.tag)
16         passages.append(node_to_passage(node, edges))
17     return passages # feeds into fuse() alongside vector and lexical hits

```

Listing 9.11: The complete, integrated graph pathway — what `graph_pathway()` now does end to end

The cross-encoder re-ranker (below) scores these graph-derived passages against the query alongside vector and lexical candidates — the best ones rise to the top 5–8 chunks sent to generation, exactly as before.

9.9 Fusion weights

Query type	Graph weight	Vector weight	Lexical weight
Factual	Low	Medium	High
Diagnostic	High	Medium	Low
Procedural	High	High	Low
Open	Low	High	Medium

Table 9.5: Query-type → pathway weight lookup table

Implementation. A simple Python dict keyed by query type maps to numeric weights applied when merging each pathway’s normalised score before re-ranking; the table itself is the entire “model” — no training loop, no persisted weights file.

9.10 The three pathways

Pathway	Implementation
Graph traversal	The five-phase context-aware pipeline above (context assembly, seed expansion, adaptive traversal, structural enrichment, passage synthesis), or the fixed-depth shallow mode (§9.14) for factual queries
Vector similarity	Qdrant search over the query embedding (same embedding model as ingestion), top 20 candidates
Lexical/BM25	Postgres FTS (<code>tsquery</code> against the <code>chunks.fts</code> column) or an in-process <code>rank_bm25</code> index, top 20 candidates

Table 9.6: The three retrieval pathways

9.11 Merge and re-ranking

Algorithm. Candidates from all three pathways are deduplicated by `chunk_id` (graph-derived synthetic passages get their own IDs), combined into a single candidate pool (target ~20–30 after dedup), and passed through a local cross-encoder (`bge-reranker` class model, running on CPU/GPU locally, no external API call) which scores each (`query`, `passage`) pair; the top 5–8 by cross-encoder score are selected for generation. **Why local, not an LLM call.** A cross-encoder re-rank is cheaper, faster, and more deterministic than scoring candidates with an LLM, and needs no extra API dependency.

9.12 Generation and citation

Prompting. The frontier-tier model is given the 5–8 selected chunks, each tagged with a stable `[doc_id:passage_id]` identifier, and instructed to cite every factual sentence with exactly one such tag, and to explicitly say “not found in the provided sources” rather than answer from parametric knowledge when the passages do not support an answer. **Citation-resolution check.** After generation, a regex extracts every `[doc_id:passage_id]` tag in the output and checks it against the set of tags that were actually provided in the prompt; any tag that does not resolve triggers exactly one regeneration attempt with an explicit reminder of the valid tag set. A second failure surfaces the answer with the unresolved sentence flagged in the UI rather than silently dropped. **Faithfulness.** Enforced by the prompt instruction above plus one self-check LLM pass on the finished answer; sentences failing the self-check are removed before the answer is returned. No separate NLI entailment model is hosted.

9.13 Interfaces

```

1  async def classify_query(query: str) -> QueryType: ...
2
3  async def retrieve(
4      query: str, query_type: QueryType, session_id: str, focused_tag: str | None,
5  ) -> list[Chunk]:
6      graph_hits, vector_hits, lexical_hits = await asyncio.gather(
7          graph_pathway(query, query_type, session_id, focused_tag),
8          vector_pathway(query),
9          lexical_pathway(query),
10     )
11     fused = fuse(graph_hits, vector_hits, lexical_hits, weights=FUSION_WEIGHTS[
12         query_type])
13     return rerank(query, fused)[:8]
14
15  async def generate_answer(query: str, chunks: list[Chunk]) -> CitedAnswer:
16      draft = await frontier_llm.generate(prompt(query, chunks))
17      if not citations_resolve(draft, chunks):
18          draft = await frontier_llm.generate(prompt(query, chunks, strict=True))
19      return self_check(draft, chunks)

```

Listing 9.12: Retrieval module interface (conceptual)

9.14 Retained fast path: shallow traversal mode

The original fixed-depth `CONNECTED_TO *1..2` query is *not* deleted. It is kept as an explicit `depth="shallow"` mode of the graph pathway, used for two purposes: (1) a low-latency fast path for factual queries, where the context-aware pipeline’s structural enrichment phase would add cost without adding relevant results, and (2) a fallback if the adaptive pipeline itself times out under its latency budget. This is why FR-RAG-10 (Chapter 3) exists as an explicit requirement, not an implicit assumption.

9.15 Edge cases

- Query mentions no resolvable entity and chat history and the focused node are both empty: seed expansion still runs Strategy B (embedding-nearest) so the pathway does not necessarily return empty, unlike the old fixed-depth-only design.
- All three pathways return low-relevance results (below a minimum score threshold): the answer explicitly states low confidence rather than forcing a citation onto a weak passage.

- A pathway times out: dropped from fusion, not blocking.
- Structural enrichment exceeds its 150ms budget ([Chapter 4](#)): dropped; adaptive traversal’s own results still proceed to passage synthesis.

9.16 Testing

Retrieval accuracy is measured against 15–20 hand-written test queries with known correct source documents (precision@5), including at least 3 queries specifically designed to exercise implicit context (no named entity) and cross-domain bridges, per [Chapter 15](#). Citation-resolution logic has unit tests covering: all tags resolve, one tag fails and regenerates successfully, one tag fails twice and is flagged. Adaptive traversal has unit tests covering: relevance decay pruning, node-budget enforcement, and edge-confidence propagation.

9.17 Performance budget

Phase	Target	Technique
Context Assembly	< 30ms	In-memory session lookup + cached entity resolution
Seed Expansion	< 80ms	Reused Qdrant search + single Postgres query
Adaptive Traversal	< 150ms	Single batched Cypher + Python scoring
Structural Enrichment	< 150ms	2–3 shortest-path queries (diagnostic/open only)
Passage Synthesis	< 20ms	Pure string formatting
Total (diagnostic/open)	< 430ms	Within the 500ms graph-pathway budget
Total (factual, shallow mode)	~ 280ms	Comparable to the original fixed-depth query

Table 9.7: Context-aware retrieval performance budget

9.18 What this does NOT require

Excluded	Why
Microsoft GraphRAG community summarisation	Requires hours of offline LLM compute to cluster and summarise — incompatible with a 5-week hackathon (Chapter 19)
Neo4j GDS Enterprise	Label propagation is approximated with the degree-based hub heuristic above; no GDS licence needed
A separate node embedding index	Qdrant chunk embeddings are reused directly, mapped to entity tags via the facts table
A new database or service	Everything above runs inside the existing FastAPI process using the existing Neo4j, Qdrant, and Postgres
Offline pre-computation	All enrichment above is query-time; no batch jobs beyond existing ingestion

Table 9.8: Deliberately excluded approaches, and their roadmap trigger

9.19 Dependencies

Neo4j, Qdrant, Postgres, the embedding model, the local cross-encoder, the fast-tier LLM (query classification), the frontier-tier LLM (generation, self-check).

9.20 Future extension

A learned, offline-trained pathway-weighting classifier is the named upgrade once enough logged query/relevance data exists ([Chapter 19](#)); a separate NLI faithfulness model, full Neo4j GDS community detection, and Microsoft GraphRAG-style offline summarisation are similar future triggers, all recorded in [Chapter 19](#).

Chapter 10

AI Models

10.1 Two-tier routing

Exactly two model tiers, called directly via their SDKs (no LiteLLM abstraction in the MVP): a fast/cheap tier for bulk extraction and classification, and a frontier tier for anything a judge or user actually reads — including compositional reasoning output.

Task	Tier	Notes
Bulk entity/relation/spec extraction	Fast/cheap	JSON-schema-constrained output against the extended predicate set (Chapter 8); one call per chunk at ingest time
Query-type classification	Fast/cheap	Single short call, or a regex/keyword pre-filter with the LLM as fallback
Copilot answer generation	Frontier	Grounding and citation faithfulness matter most
CORTEX Diagnose / Comply synthesis	Frontier	Multi-document reasoning benefits materially from a stronger model; call volume is low
CORTEX Asset compositional reasoning (constraint extraction, compatibility filtering, justified selection)	Frontier	Multi-attribute logical reasoning over structured specs; the same model tier already used for Diagnose/Comply, invoked via a different tool set and prompt (Chapter 11)
P&ID structured extraction	Frontier, multimodal	One call per drawing image
Re-ranking	Local cross-encoder (no LLM call)	Cheaper and more deterministic
Voice transcription (optional)	Whisper API / Web Speech API	Only if the voice-input stretch goal is reached

Table 10.1: Task-to-model mapping

10.2 Why every model exists

- **Fast-tier model.** Keeps ingestion cost and latency bounded across hundreds of documents and thousands of chunks, whether extracting a relational fact or a specification fact.
- **Frontier-tier model.** The visible product — Copilot answers, agent conclusions, and compositional recommendations — is judged directly on grounding quality and reasoning correctness; this is not a place to economise.
- **Frontier multimodal.** P&ID symbol/adjacency extraction needs vision understanding a fast-tier text model cannot provide.
- **Local cross-encoder.** Avoids paying LLM latency/cost for a task a small, purpose-built model does deterministically and fast.
- **Embedding model.** One model, both query and document side, reused for vector retrieval, graph seed expansion, and entity resolution’s embedding-fallback stage.

10.3 Expected input/output per task

Task	Input	Output
Bulk extraction	One chunk of parsed text + extended ontology schema	JSON list of {subject, predicate, object, confidence} triples, predicate drawn from the extended set
Query classification	Raw query string	One of {factual, diagnostic, procedural, open}
Copilot generation	Query + 5–8 ranked chunks with tags	Cited prose answer
Agent synthesis (Diagnose/Comply)	Graph query results + retrieved passages	Structured hypothesis/score + narrative explanation
CORTEX Asset compositional reasoning	Target entity’s specs + candidate entities’ specs	Constraint list, per-candidate pass/fail table, cited recommendation
P&ID extraction	Drawing image	JSON {symbols[], adjacency_pairs[]}

Table 10.2: Expected input/output shape per task

10.4 Prompt strategy

All structured-output prompts follow the same shape: (1) a system instruction naming the exact JSON schema and forbidding prose outside it; (2) the extended ontology or tag set as in-context reference data, never left to the model’s parametric knowledge; (3) an explicit instruction to omit rather than guess when a field cannot be determined from the given text; (4) for generation tasks, an explicit citation-tag instruction and an explicit “say so if unsupported” instruction ([Chapter 9](#)); (5) for compositional reasoning specifically, an explicit instruction to never guess a specification value and to state “not found in available documents” rather than assume a default ([Chapter 11](#), §11.5.4).

10.5 JSON schema example: bulk extraction (extended)

```

1 {
2   "type": "object",
3   "properties": {
4     "triples": {
5       "type": "array",
6       "items": {
7         "type": "object",
8         "properties": {
9           "subject": {"type": "string"},
10          "predicate": {"type": "string", "enum": [
11            "experiencedFailure", "hasProcedure", "connectedTo",
12            "regulatedBy", "addresses", "usesPart", "mentions",
13            "hasSpec", "hasMaterial", "operatingTemp",
14            "operatingPressure", "fitsRange", "compatibleWith", "ratedFor"]},
15          "object": {"type": "string"},
16          "confidence": {"type": "number", "minimum": 0, "maximum": 1}
17        },
18        "required": ["subject", "predicate", "object", "confidence"]
19      },
20    },
21  },
22  "required": ["triples"]
23 }

```

Listing 10.1: Structured extraction output schema, extended with specification-type predicates

10.6 Does compositional reasoning need fine-tuning?

No — not for the MVP. Every capability CORTEX Asset’s compositional reasoning mode needs is already within reach of a frontier general-purpose model, zero-shot. The real bottleneck is extraction coverage, not reasoning sophistication: see the warning box at the end of this section, and [Chapter 11 §11.5.4](#) for how this plays out inside the agent.

Capability	Zero-shot capable?	Notes
Decompose a query into spec dimensions	Yes	Basic engineering reasoning
Extract structured values from spec tables	Yes	Already used in extraction (Chapter 6)
Compare spec sets for compatibility	Yes	Logical comparison — frontier LLMs are strong at this
Common material–chemical compatibility	Mostly	Common pairs are well represented in training data; obscure ones may not be
Company-specific selection rules	No	Institutional knowledge that was never in training data and is not in the demo corpus either

Table 10.3: Zero-shot readiness of compositional reasoning capabilities

Problem (post-MVP)	MVP alternative (no fine-tuning)
Company-specific selection rules	Encode as facts in the KG via a <code>selectionRule</code> relationship type (Chapter 19)
Material compatibility hallucination	Add a compatibility lookup table as an agent tool (Chapter 19)
Industrial spec-format misreads (“4in” vs “DN100”)	Normalise specs at extraction time (preprocessing, not model-level)

Table 10.4: Where fine-tuning would help post-MVP, and the MVP alternative used instead

The real bottleneck is extraction, not reasoning. If the ingestion pipeline (Chapter 6) never extracts “P-301 has inner diameter 4 inches” as a structured specification fact, no amount of query-time reasoning sophistication in CORTEX Asset will find it. The single highest-impact change made for compositional reasoning is enriching the extraction predicate set (Chapter 6), not the reasoning chain itself (Chapter 11, §11.5.4).

10.7 Failure handling

Malformed JSON output is retried once with a stricter schema reminder; persistent malformed output is logged and the chunk/document skipped rather than blocking the pipeline. Frontier-tier timeouts on Copilot generation, or on compositional reasoning steps, retry once before surfacing a degraded response (Chapter 4).

10.8 Cost and latency management

Bulk extraction cost scales with chunk count, not document count. Adding specification-type predicates to the extraction schema does not change this scaling — it is the same one call per chunk, now capable of returning a richer set of triples per call, so the extended predicate set is essentially free in terms of extraction cost.

10.9 Dependencies

Direct SDK access to one fast-tier and one frontier-tier LLM provider, one multimodal-capable frontier model, one open embedding model (or provider embedding endpoint), one local cross-encoder checkpoint.

10.10 Future extension

LiteLLM abstraction for provider-agnostic routing, and a third/fourth tier, are named *Could*/roadmap items (Chapter 19).

Chapter 11

Agent Design

11.1 Pattern: LangGraph Supervisor $\rightarrow$ three workers

The multi-agent system is built with LangGraph’s native supervisor–worker pattern, exactly three workers, unchanged in count and shape by the addition of compositional reasoning: the four-worker v2 design (Maintenance, RCA, Compliance, Lessons-Learned) was merged to three early in design, since Maintenance and Lessons-Learned both mine work-order/failure history over the same underlying graph query, one framed forward and one framed backward.

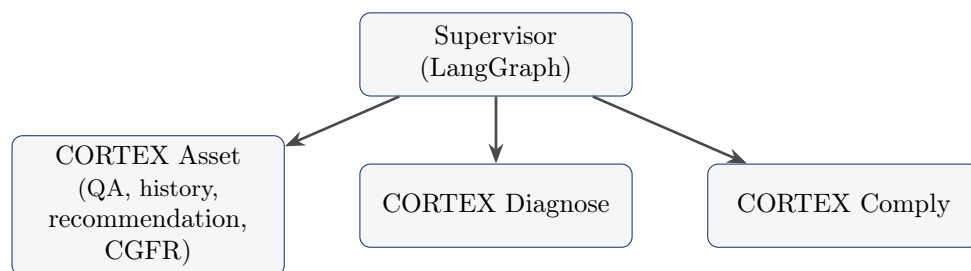


Figure 11.1: Supervisor–worker agent architecture — unchanged shape; CORTEX Asset’s tool set is extended, no fourth worker is introduced

11.2 Table of workers

Engineering name	Product name	Job
Asset Intelligence (merged Maintenance + Lessons-Learned; now also Compositional Reasoning)	CORTEX ASSET	Mines work-order and failure history for a given tag; surfaces recurring failure modes, frequency-based risk scoring, pattern-matches against similar past incidents; <i>and</i> answers component-selection / compatibility / specification-comparison questions via the CGFR reasoning pattern (§11.5.4)
Root Cause Analysis	CORTEX DIAGNOSE	Plan-then-execute: pulls depth-bounded equipment context, generates ranked causal hypotheses with evidence citations, then self-critiques and discards unsupported hypotheses
Compliance	CORTEX COMPLY	Checks ~10–15 curated regulation clauses against equipment inspection/procedure facts; flags gaps and names missing evidence

Table 11.1: The three CORTEX workers

The Copilot (Chapter 9) is not a fourth worker; it remains the default query path (**CORTEX Ask**).

11.3 Supervisor: responsibilities and decision flow

Purpose. Route an incoming query or explicit invocation to exactly one worker, or to no worker (handled directly by Copilot). **Decision flow.**

1. Explicit invocation (e.g. `POST /api/agents/diagnose`) routes directly to the named worker, bypassing routing logic.
2. An implicit trigger from a Copilot query classified diagnostic and matching a root-cause pattern routes to **CORTEX DIAGNOSE** asynchronously.
3. A query about historical patterns, forward risk, *or* component selection/compatibility/specification comparison routes to **CORTEX ASSET** — the same worker handles all four, distinguished only by which internal tools and prompt branch it invokes (§11.5).
4. A query naming a regulation or asking about compliance status routes to **CORTEX ComPLY**.
5. No match: Copilot answers directly, no worker invoked.

Implementation. A LangGraph graph with the Supervisor as the entry node, using a single fast-tier LLM routing call (given the query, the query classification from Chapter 9, and short worker descriptions — including a description of CORTEX Asset’s compositional-reasoning capability) to pick the destination node, or a rule match on the explicit-invocation endpoints to skip the LLM call entirely.

11.4 Autonomy tiers

Tier	Behaviour
Tier 1 — Respond	Read-only; answers, hypotheses, and compositional recommendations shown directly to the user; logged to <code>audit_log</code> .
Tier 2 — Propose	A maintenance-hold suggestion or a compliance-gap flag is shown in the UI with a “proposed — requires review” badge and a confirm/dismiss button, gated by role. No auto-execute tier exists in the MVP; a compositional recommendation is always Tier 1, never auto-ordered or auto-procured.

Table 11.2: Two autonomy tiers, no policy-engine microservice

Each worker is capped at a small fixed number of tool calls, reducing the entire “circuit breaker” concern to a single `max_iterations` parameter in `LangGraph`.

11.5 CORTEX Asset

11.5.1 Four capabilities, one worker

CORTEX Asset supports four distinct question shapes without becoming four different agents:

1. **Normal QA / history.** “What has failed on P-101A before?” — the original failure-history and risk-scoring behaviour.
2. **Recommendation generation.** “What should we watch for on P-101A?” — the forward-framed narration of the same failure-history query.
3. **Compatibility analysis and specification comparison.** “Is F-42 compatible with P-301?” / “Compare F-42 and F-44 for pipe P-301.” — structured multi-attribute comparison via the tools in §11.5.3.
4. **Compositional reasoning / component selection.** “Which funnel should I use for pipe P-301?” — the full Constraint-Gather-Filter-Reason chain, §11.5.4.

All four share the same `LangGraph` node, the same tool registry, and the same `max_iterations` cap; only the prompt branch and the subset of tools invoked differ, selected by the Supervisor’s routing signal and, within CORTEX Asset itself, by a lightweight intent check on the query.

11.5.2 Prompt strategy for capabilities 1–2 (QA/history)

Given a tag, the worker’s `LangGraph` node runs the Cypher failure-history query (Chapter 8), then prompts the frontier model to narrate the result twice: once framed forward (“based on this pattern, what should we watch for”) and once backward (“here is what has happened before on similar equipment”). **Tool calling.** `graph_query(tag)` (the enhanced, context-aware `context_graph_query` described in Chapter 9); `max_iterations=3`. **Failure recovery.** If the graph query returns no history (a genuinely new asset), the worker returns an explicit “no history available” response rather than fabricating a pattern.

11.5.3 Tools

```

1 AGENT_TOOLS = [
2     # History / QA (existing)
3     context_graph_query, # context-aware graph query, Ch. 9 -- shallow or deep
4     retrieve,           # vector + lexical search, Ch. 9

```

```

5
6     # Compatibility analysis and compositional reasoning
7     get_entity_specs,          # pull all spec-type facts for one entity
8     find_by_type,             # find all entities of a given type/subtype, with
                               specs
9     compare_specs,            # structured pairwise comparison of two entities'
                               specs
10 ]

```

Listing 11.1: CORTEX Asset’s tool registry — history/QA tools plus the tools added for compatibility analysis and compositional reasoning

```

1  async def get_entity_specs(tag: str) -> dict:
2      """Pull all spec-type facts for an entity (graph + vector)."""
3      graph_specs = await neo4j.run("""
4          MATCH (e {tag: $tag})-[r]->(v)
5          WHERE type(r) IN ['hasSpec','hasMaterial','operatingTemp',
6                          'operatingPressure','fitsRange','ratedFor']
7          RETURN type(r) AS predicate, COALESCE(v.value, v.tag) AS value,
8                  r.confidence AS confidence, r.source_doc_id AS source
9      """, tag=tag)
10     passages = await qdrant_search(f"{tag} specifications dimensions material",
11                                   top_k=5)
12     return {"tag": tag, "structured_specs": graph_specs, "supporting_passages":
13           passages}

```

Listing 11.2: `get_entity_specs` – pulls structured specification facts plus supporting passages for one entity

```

1  async def find_by_type(entity_type: str, subtype: str | None) -> list[dict]:
2      """Find all entities of a given type with their specs."""
3      candidates = await neo4j.run("""
4          MATCH (e:{entity_type}) WHERE ($subtype IS NULL OR e.type = $subtype)
5          RETURN e.tag, e.attributes, e.type LIMIT 20
6      """, subtype=subtype)
7      return [{"tag": c["tag"], "specs": await get_entity_specs(c["tag"])} for c in candidates]

```

Listing 11.3: `find_by_type` – finds candidate components of a given type/subtype, with specs attached

```

1  async def compare_specs(target_tag: str, candidate_tag: str) -> dict:
2      """Structured comparison of two entities' specifications, dimension by
3      dimension."""
4      target = await get_entity_specs(target_tag)
5      candidate = await get_entity_specs(candidate_tag)
6      return build_comparison_table(target, candidate) # property-by-property
7      pass/fail

```

Listing 11.4: `compare_specs` – structured pairwise comparison, used both standalone (capability 3) and inside CGFR (capability 4)

11.5.4 Compositional Reasoning: the Constraint-Gather-Filter-Reason (CGFR) pattern

Why this exists

Some of the most valuable industrial questions have answers that were never written down in any single document. “Which funnel should I use for pipe P-301?” is never answered by a sentence like “use funnel F-42 for pipe P-301” anywhere in the corpus — that sentence does not exist. The answer is only *derivable* from specification facts scattered across multiple documents (an OEM manual, a P&ID drawing, a process data sheet, a spare parts catalog).

No embedding search, no BM25 match, and no graph traversal — however context-aware — will find a single passage that states the answer, because there is no edge in the graph between P-301 and any funnel; they were never mentioned together. The system must *construct* the answer from parts. This is fundamentally a reasoning workflow, not a retrieval pathway, and it runs strictly *after* evidence retrieval: the flow for these queries is **User Query** → **Query Classification** → **Retrieval** → **Evidence Fusion** → **Reasoning**, with CGFR as one reasoning strategy CORTEX Asset selects only for this query class.

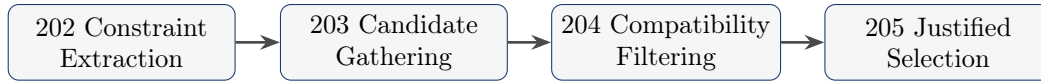


Figure 11.2: The four-step Constraint-Gather-Filter-Reason (CGFR) pattern

Step 1 — Constraint Extraction (pure LLM reasoning, no retrieval). The LLM decomposes the original query into the specification dimensions that constrain the answer, before any tool is called.

```

1 Input: "Which funnel should I use for pipe P-301?"
2 LLM ->: Need: diameter (fit), material (compatibility),
3         operating temp (thermal rating), operating pressure (pressure rating),
4         fluid/medium (corrosion compatibility)
  
```

Listing 11.5: Constraint extraction example

This step relies on ordinary engineering domain knowledge already present in a frontier general-purpose model (Chapter 10); no fine-tuning is required.

Step 2 — Candidate Gathering (two parallel retrievals via existing tools). `get_entity_specs(target)` pulls the target equipment's specification facts; `find_by_type(entity_type, subtype)` pulls all candidate components of the relevant type, each with its own specs attached. Both tools also run a vector search (Chapter 9) alongside the graph query, to catch specification passages the graph missed.

Step 3 — Compatibility Filtering (LLM multi-attribute reasoning). The LLM receives the target's extracted constraints and every candidate's specifications, and evaluates compatibility across every dimension explicitly:

```

1 P-301: diameter=4", material=316SS, temp=150C, pressure=10bar
2
3 F-42: fits 3"-6", 316SS, 200C, 15bar -> COMPATIBLE (all 4 dimensions pass)
4 F-19: fits 3"-5", PTFE, 250C, 8bar -> REJECTED (pressure: 8 < 10 bar)
5 F-55: fits 6"-10", Carbon Steel, 300C -> REJECTED (diameter: 6" min > 4" pipe)
  
```

Listing 11.6: Compatibility filtering example

Step 4 — Justified Selection (cited answer). The LLM produces a recommendation where every claim traces to a specific fact:

Example output

Recommended: F-42 — Fits 4" diameter (range 3"-6") [Catalog:p8], material match 316SS

[OEM Manual:p12, Catalog:p8], rated 200°C > 150°C operating [Process Data:r47, Catalog:p9], rated 15 bar > 10 bar [Catalog:p9].

11.5.5 Agent prompt addition for CGFR

```

1 When a user asks about component selection or "which X for Y" questions:
2 1. DECOMPOSE: Identify relevant spec dimensions (size, material, temp, pressure
  ... )
3 2. GATHER: Use get_entity_specs() for the target; find_by_type() for candidates
4 3. EVALUATE: Check each candidate on every dimension -- explicit pass/fail
5 4. RECOMMEND: Best match with full citation chain. If none match, say so
  explicitly.
6 6. CRITICAL: Never guess a spec. If not in retrieved data, say "not found."
```

Listing 11.7: Prompt branch added to CORTEX Asset for component-selection / compatibility questions

11.5.6 Edge case handling

Situation	Behaviour
No candidates of the required type exist in the corpus	CORTEX Asset states the target's known specs, states the requirements a compatible part would need to meet, and explicitly flags the recommendation as based on general engineering principles rather than corpus documents
Target specs incomplete (e.g. material missing)	CORTEX Asset reports what it found with citations, explicitly flags the missing dimension as "not found," and states which conclusions can and cannot yet be drawn
Multiple candidates pass every constraint	All are presented in a structured comparison table (FR-UI-6, Chapter 3) with any relevant trade-offs (e.g. wider safety margin) named, rather than an arbitrary single pick
Zero candidates pass every constraint	CORTEX Asset states this explicitly and names which constraint is hardest to satisfy, rather than recommending the closest partial match silently

Table 11.3: CGFR edge case handling

11.6 CORTEX Diagnose

Decision flow (plan-then-execute).

1. **Plan.** Given the query and the target equipment tag, generate a short plan: which graph neighbourhoods and document types to pull as context.
2. **Execute.** Pull context-aware equipment context (adaptive traversal, [Chapter 9](#), invoked in "deep" mode) plus retrieved passages for the plan's targets.
3. **Hypothesize.** Generate ranked causal hypotheses, each with an evidence citation.
4. **Self-critique.** A second frontier-tier pass reviews each hypothesis against its cited evidence and discards any hypothesis not actually supported.

5. **Stream.** Each step is streamed as an SSE event and logged with a sequence number, enabling Replay Reasoning (Chapter 13).

Tool calling. `context_graph_query(tag, query, depth="deep"), retrieve(query); max_iterations=5.`

Failure recovery. If self-critique discards every hypothesis, the worker returns an explicit “insufficient evidence for a confident root cause” response.

11.7 CORTEX Comply

Decision flow. For each of the curated 10–15 regulation clauses relevant to the equipment scope: run the compliance gap Cypher query (Chapter 8); for each gap, resolve the specific missing evidence type; aggregate into a single score and a red/amber/green list. **Tool calling.** `context_graph_query(reg_code, depth="shallow"); max_iterations=3.` **Failure recovery.** A regulation with no equipment in scope is skipped from the score denominator, not counted as a false gap.

11.8 The enhanced context_graph_query tool

All three workers reach the graph through one enhanced tool, which internally dispatches to either the shallow fixed-depth mode or the full five-phase context-aware pipeline (Chapter 9):

```

1  async def context_graph_query(
2      tag: str, query: str, depth: str = "auto", include_analogues: bool = False,
3  ) -> GraphContext:
4      """Enhanced graph query tool for agents.
5
6      'shallow': fixed-depth traversal only, max_depth=2 (fast, ~100ms) -- Ch. 9
7                  Sec. Retained
8                  Fast Path.
9      'deep':     full five-phase context-aware pipeline, max_depth=5 (~300-500ms)
10                 -- Ch. 9.
11      'auto':     shallow for Asset (history mode) / Comply, deep for Diagnose.
12      """
13      ...

```

Listing 11.8: `context_graph_query` – the single graph-access tool shared by all three workers

This means CORTEX Diagnose’s plan-then-execute flow automatically gets richer context without changing its prompts — it just gets better passages from the same tool call; CORTEX Asset and CORTEX Comply automatically get a fast shallow query for their typical single-equipment lookups, with the option to request `depth="deep"` explicitly for a diagnostic-flavoured Asset question.

11.9 Prompts (representative)

```

1  System: You are CORTEX Diagnose, a root-cause reasoning agent for industrial
2         equipment.
3  Given the equipment context and retrieved passages below, generate 2-4 ranked
4         causal
5         hypotheses. Each hypothesis MUST cite a [doc_id:passage_id] or a graph fact_id.
6         If the
7         evidence does not support a confident hypothesis, say so explicitly. Do not
8         speculate
9         beyond what the provided context supports.
10
11 Equipment context: {graph_context}

```

```

8 Retrieved passages: {passages}
9 Query: {query}

```

Listing 11.9: CORTEX Diagnose hypothesis-generation prompt skeleton

11.10 Interfaces

```

1 class SupervisorState(TypedDict):
2     query: str
3     query_type: QueryType
4     route: Literal["asset", "diagnose", "comply", "none"]
5
6 async def route(state: SupervisorState) -> SupervisorState: ...
7
8 async def cortex_asset(tag: str, query: str) -> AssetResult:
9     # dispatches internally to history/QA, compatibility, or CGFR based on intent
10    ...
11 async def cortex_diagnose(tag: str, query: str) -> DiagnoseResult: ... # streams
12    steps
13 async def cortex_comply(scope: str | None) -> ComplyResult: ...

```

Listing 11.10: Supervisor and worker interface (conceptual)

11.11 Logging

Every worker invocation writes one `audit_log` row per step (plan, tool call, hypothesis or constraint, evaluation verdict, final answer), with a monotonic sequence number per session — this sequence is exactly what the Replay Reasoning UI control replays ([Chapter 13](#)), including CGFR’s four-step chain.

11.12 Testing

Agent Success Rate ([Chapter 15](#)) is measured as the percentage of scripted invocations that complete within the iteration cap and produce an evidence-linked answer, across all four CORTEX Asset capabilities. Unit tests cover routing decisions, per-worker tool-call sequencing, and — specifically for CGFR — constraint-extraction correctness on a fixed set of sample queries, and compatibility-filtering correctness on a fixed set of target/candidate spec pairs with known pass/fail outcomes.

11.13 Expected latency

CORTEX Asset (history/QA/recommendation): < 15s P95. CORTEX Asset (compatibility analysis, single pairwise `compare_specs`): < 15s P95. CORTEX Asset (CGFR, compositional): < 25s P95. CORTEX Comply: < 15s P95. CORTEX Diagnose: < 30s P95 ([Chapter 4](#)).

11.14 Dependencies

LangGraph, the frontier-tier LLM, Neo4j (via `context_graph_query`), the retrieval module ([Chapter 9](#)), Postgres `audit_log`.

11.15 Future extension

A third autonomy tier with an OPA policy engine, a fourth worker if a genuinely distinct job emerges, a company-specific **selectionRule** relationship type, and a material compatibility lookup tool to reduce reliance on parametric chemistry knowledge, are named roadmap items (Chapter 19); no such addition is in scope for this build.

Chapter 12

APIs

12.1 Endpoint surface

Endpoint names follow the product branding rather than internal engineering labels. This is the complete REST surface; no endpoint outside this list exists in the MVP. Compositional reasoning and context-aware traversal introduce *no new endpoints* — both are reached through the existing `/api/query` and `/api/agents/*` surface.

```
1 POST    /api/documents          # upload + ingest
2 GET     /api/documents/{id}    # status
3 POST    /api/query             # CORTEX Ask query (SSE stream
    supported)
4 GET     /api/entities/{tag}    # entity detail
5 GET     /api/entities/{tag}/graph?depth=2 # entity detail
6 POST    /api/entities/{tag}/feedback # human correction -> new fact
7 POST    /api/agents/diagnose    # async CORTEX Diagnose (RCA) session
8 GET     /api/agents/diagnose/{job_id}
9 GET     /api/agents/diagnose/{job_id}/replay # replay the streamed reasoning steps
10 GET    /api/agents/comply/gaps  # CORTEX Comply gap report / score
11 GET    /api/agents/asset/risk/{tag} # CORTEX Asset: forward risk scoring
12 GET    /api/agents/asset/history/{tag} # CORTEX Asset: pattern-matched past
    incidents
13 POST    /api/agents/asset/recommend # CORTEX Asset: compositional
    recommendation
14                                     # (CGFR), body: {query, target_tag}
15 WS      /ws/copilot/{session_id}  # streaming answer + agent follow-up
```

Listing 12.1: Complete API surface

12.2 Request/response examples

12.2.1 POST /api/documents

```
1 // multipart/form-data: file=<binary>, authority_class=Supporting
```

Listing 12.2: Request

```
1 {"doc_id": "b3f1...", "status": "queued", "status_url": "/api/documents/b3f1..."}
```

Listing 12.3: Response 202

12.2.2 POST /api/query

```
1 {"query": "Why does P-101A keep failing?", "session_id": "s-88a2", "focused_tag":  
2   "P-101A"}
```

Listing 12.4: Request, with the new focused_tag field

```
1 event: token  
2 data: {"text": "Pump P-101A has experienced "  
3  
4 event: citation  
5 data: {"doc_id": "d-91", "passage_id": "p-4", "page": 3}  
6  
7 event: agent_trigger  
8 data: {"worker": "diagnose", "job_id": "j-501"}  
9  
10 event: done  
11 data: {"answer_id": "a-771"}
```

Listing 12.5: SSE stream events

12.2.3 GET /api/entities/{tag}/graph?depth=2

```
1 {  
2   "center": "P-101A",  
3   "nodes": [{ "tag": "P-101A", "type": "Equipment"}, { "tag": "P-101B", "type": "  
4     Equipment"}],  
5   "edges": [{ "source": "P-101A", "target": "P-101B", "type": "CONNECTED_TO", "  
6     confidence": 0.55}]  
7 }
```

Listing 12.6: Response 200

12.2.4 POST /api/entities/{tag}/feedback

```
1 {"predicate": "experiencedFailure", "object": "Misalignment", "note": "Confirmed  
2   on-site"}
```

Listing 12.7: Request

```
1 {"fact_id": "f-9902", "status": "active", "supersedes": "f-8811"}
```

Listing 12.8: Response 201

12.2.5 POST /api/agents/diagnose

```
1 {"tag": "P-101A", "query": "root cause of recurring bearing failure"}
```

Listing 12.9: Request

```
1 {"job_id": "j-501", "status": "running"}
```

Listing 12.10: Response 202

12.2.6 GET /api/agents/diagnose/{job_id}

```

1 {
2   "job_id": "j-501", "status": "complete",
3   "hypotheses": [
4     {"rank": 1, "cause": "Shaft misalignment", "confidence": 0.78,
5      "evidence": ["d-91:p-4", "fact:f-7743"]}
6   ]
7 }

```

Listing 12.11: Response 200 (completed)

12.2.7 GET /api/agents/comply/gaps

```

1 {
2   "score": 0.86, "checked": 15, "missing": 2,
3   "gaps": [{"equipment": "P-101A", "regulation": "OISD-154", "missing_evidence":
4     "recertification record"}]

```

Listing 12.12: Response 200

12.2.8 POST /api/agents/asset/recommend (compositional reasoning)

```

1 {"query": "Which funnel should I use for pipe P-301?", "target_tag": "P-301"}

```

Listing 12.13: Request

```

1 {
2   "recommendation": "F-42",
3   "constraints": ["diameter", "material", "operatingTemp", "operatingPressure"],
4   "candidates_evaluated": [
5     {"tag": "F-42", "verdict": "compatible", "checks": {"diameter": true, "
6       material": true,
7       "operatingTemp": true, "operatingPressure": true}},
8     {"tag": "F-19", "verdict": "rejected", "checks": {"diameter": true, "material":
9       true,
10      "operatingTemp": true, "operatingPressure": false}, "reject_reason": "
11      operatingPressure"},
12     {"tag": "F-55", "verdict": "rejected", "checks": {"diameter": false},
13      "reject_reason": "diameter"}
14   ],
15   "citations": ["Catalog:p8", "Catalog:p9", "OEM Manual:p12", "Process Data:r47"]
16 }

```

Listing 12.14: Response 200

12.3 Error format

All error responses share one envelope so the frontend needs one error-handling path:

```

1 {"error": {"code": "ENTITY_NOT_FOUND", "message": "Tag P-999Z not found", "detail":
2   null}}

```

Listing 12.15: Standard error envelope

A CGFR request that finds zero compatible candidates returns 200, not an error — “no compatible candidate found” is a valid, explicitly-stated answer (Chapter 11, §11.5.4), not a failure of the API.

12.4 Streaming transport

`/api/query` uses SSE for one-directional streaming (token, citation, `agent_trigger`, done events).

`/ws/copilot/{session_id}addsbidirectionalinteraction.Bothtransportssharethesameeventschema./api/agen`

12.5 Versioning

All endpoints are implicitly v1; no version prefix exists in the MVP.

12.6 Rate limiting and auth

Every endpoint except health checks requires a valid JWT ([Chapter 14](#)); no rate limiting in the MVP.

12.7 Testing

Contract and integration tests per endpoint; see [Chapter 16](#). FastAPI's auto-generated OpenAPI schema (`/docs`) is the live contract reference.

Chapter 13

Frontend

13.1 Graph-first navigation

The graph explorer is the spine every other feature hangs off. Search always resolves to a node; every node is the entry point to its documents, its Copilot context, and its agents; every agent conclusion -- including a CGFR recommendation -- writes back onto the graph as a highlighted path. Selecting a node also sets `focused_tag` for every subsequent query in the session (Chapter 9, §9.4), so the graph explorer is not just a visualisation surface but an active input to retrieval.

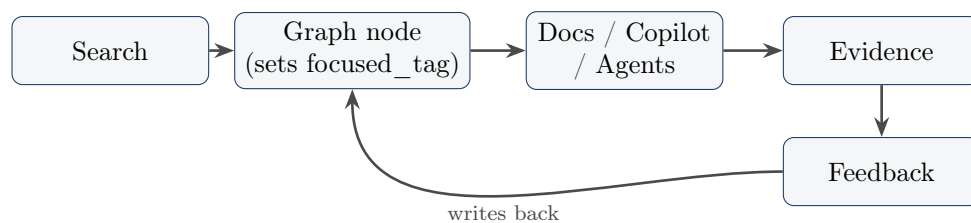


Figure 13.1: Graph-first navigation loop, including the `focused_tag` feedback into retrieval

13.2 Page-by-page

Page/route	Content
/login	Role-based login; seeds one of 4 demo accounts.
/ (dashboard)	Opens directly on the graph explorer (Cytoscape.js); a morning-alert banner highlights any equipment matching a pre-failure pattern (CORTEX Asset).
/graph	Full-screen graph explorer with depth control and click-to-expand; the primary navigation surface; selecting a node sets <code>focused_tag</code> .
/entity/[tag]	Side panel (or full page on mobile) with CORTEX Ask chat, maintenance timeline, linked documents, and agent invocation buttons — including “Ask CORTEX Asset which part fits” for compositional queries — for the selected tag.
/compliance	Compliance score view: single score, red/amber/green equipment list, click-through to missing evidence on the graph.
/agents/diagnose/[jobId]	Live streaming reasoning overlay; Replay Reasoning control once complete.
/agents/asset/recommend	Live streaming CGFR reasoning overlay (constraint extraction, candidate gathering, compatibility filtering, justified selection), rendering the final comparison as a structured table (FR-UI-6).

Table 13.1: Page-by-page breakdown

13.3 Component hierarchy

```

1 <App>
2   <AuthProvider>
3     <Layout>
4       <GraphExplorer>           // Cytoscape.js wrapper; tracks focusedTag in
          GraphStore
5       <NodeSidePanel>
6         <CopilotChat />         // streaming SSE/WS client, sends focused_tag
7         <MaintenanceTimeline />
8         <LinkedDocuments />
9         <AgentInvokeBar>
10        <AssetPanel />          // history / recommendation / compatibility /
          CGFR
11        <DiagnosePanel /> <ComplyPanel />
12      </AgentInvokeBar>
13    </NodeSidePanel>
14  </GraphExplorer>
15  <ComplianceScoreView />
16  <ReasoningOverlay>           // live streaming + Replay control (RCA and CGFR
          steps)
17    <ReplayControls />
18    <ComparisonTable />        // structured pass/fail table for CGFR results
19  </ReasoningOverlay>
20 </Layout>
21 </AuthProvider>
22 </App>

```

Listing 13.1: Top-level component tree (Next.js + React)

13.4 State management

Session/auth state (JWT, role) lives in a top-level context provider. Graph state (currently centred node, expanded depth, highlighted paths, and *the currently focused tag*) lives in a dedicated GraphStore so both the explorer and the side panel can react to the same selection without prop drilling, and so `focused_tag` can be read by the query client on every Copilot request. Streaming state (in-flight tokens, citations, agent steps, including CGFR’s four-step chain) is buffered in a per-session reducer keyed by `session_id/job_id`, appended to as SSE/WS events arrive, and flushed to the UI on each animation frame. No browser storage (`localStorage/sessionStorage`) is used; everything lives in memory for the session.

13.5 Streaming UI

Copilot tokens render incrementally as they arrive over SSE; each citation event attaches a small citation chip to the sentence being generated, clickable to open the source passage with its bounding box highlighted. Agent reasoning events (plan, hypothesis, critique, and -- for CORTEX Asset’s compositional mode -- constraint, candidate, compatibility recommendation) render as a vertically stacked timeline in the Reasoning Overlay, each entry linking back to a highlighted graph node or document.

13.6 Graph animations

Node selection triggers a Cytoscape.js layout re-centre and a staged reveal: connected documents and sister equipment fade in over ~400ms, followed by the maintenance timeline sliding in from the side panel, followed by any active failure-pattern edges pulsing to draw attention. Edges below a 0.7 confidence threshold (Chapter 8) render dashed.

13.7 Replay Reasoning

Because every CORTEX Diagnose step and every CGFR step is already streamed and logged with a sequence number (Chapter 11), the Replay control simply re-fetches the logged sequence via the replay endpoint (Chapter 12) and re-plays the same events and graph highlights at a controlled pace client-side -- no new backend logic beyond the storage the streaming overlay already produces.

13.8 Compositional comparison tables

CORTEX Asset’s compatibility and CGFR answers render as a structured, property-by-property comparison table (target requirement vs. each candidate’s value, with a pass/fail indicator per cell) rather than as unstructured prose, per FR-UI-6. This is the same `ComparisonTable` component used for both a standalone `compare_specs` call and the final step of a full CGFR chain.

13.9 Compliance, made visual

A single score, a colour-coded equipment list, and click-through from any red item straight to the missing evidence on the graph.

13.10 Mobile responsiveness

A responsive layout that works acceptably on a phone browser, with a voice-input button (stretch goal) for the Copilot. No offline caching, service-worker sync, or CRDT conflict resolution.

13.11 Testing

Component tests cover the streaming reducer, the graph store (selection, expansion, highlight state, and focusedTag propagation), and the ComparisonTable component's pass/fail rendering against fixed sample data. A Playwright/Cypress end-to-end test drives the exact demo script ([Chapter 18](#)) headlessly as a regression check before each rehearsal.

13.12 Expected latency

Perceived responsiveness targets: first graph paint < 1s, node-click reveal animation < 1.5s total, Copilot first token per [Chapter 4](#).

13.13 Dependencies

Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, Cytoscape.js.

13.14 Future extension

Offline-first PWA with CRDT sync and a native mobile app are named roadmap items ([Chapter 19](#)).

Chapter 14

Authentication

14.1 Purpose and scope

Demonstrate the access-control concept the problem statement cares about, without standing up an identity provider or a policy engine for a prototype nobody will penetration-test. Simple JWT auth with 3-4 hardcoded demo roles and role-based (not attribute-based) route guards.

14.2 Roles

Role	Typical access
Technician	Copilot (CORTEX Ask), graph explorer (read), maintenance timeline, CORTEX Asset history/QA/recommendation queries; cannot confirm Tier-2 proposals or view Personnel names.
Engineer	All technician access plus CORTEX Diagnose invocation, CORTEX Asset compatibility/CGFR queries, Tier-2 confirm/dismiss for maintenance actions.
Compliance Officer	All engineer access plus CORTEX Comply, compliance score view, regulation gap detail.
Admin	Full access, including <code>audit_log</code> view and user management (seed accounts only in the MVP).

Table 14.1: The four seeded demo roles

14.3 JWT flow

1. User submits username/password to `POST /api/auth/login`.
2. Backend verifies the password hash (`users.password_hash`, `bcrypt`) and issues a signed JWT with claims `{sub: user id, role, exp}`.
3. Frontend stores the JWT in memory and attaches it as a Bearer token on every subsequent request.
4. A FastAPI dependency validates signature and expiry on every protected route, and injects the decoded role claim into the request context for the route guard to check.
5. On expiry, the frontend redirects to `/login`; there is no refresh-token flow in the MVP.

```

1 def require_role(*allowed: str):
2     def dependency(claims: dict = Depends(decode_jwt)):
3         if claims["role"] not in allowed:
4             raise HTTPException(403, detail={"code": "FORBIDDEN_ROLE"})
5         return claims
6     return dependency
7
8 @app.post("/api/agents/asset/recommend")
9 async def asset_recommend(claims=Depends(require_role("engineer", "
10     compliance_officer", "admin"))):
11     ...

```

Listing 14.1: Route guard dependency (conceptual)

14.4 Permissions matrix

Endpoint	Technician	Engineer	Compliance	Admin
/api/query	✓	✓	✓	✓
/api/entities/*	✓	✓	✓	✓
/api/entities/*/feedback		✓	✓	✓
/api/agents/diagnose		✓	✓	✓
/api/agents/asset/history, /risk	✓	✓	✓	✓
/api/agents/asset/recommend (CGFR)		✓	✓	✓
/api/agents/comply/*			✓	✓
Tier-2 confirm/dismiss		✓	✓	✓
audit_log view				✓

Table 14.2: Coarse RBAC permissions matrix

14.5 Edge cases

- Expired token mid-stream (during an SSE session, including a CGFR reasoning stream): the stream is closed with a 401-equivalent SSE error event; the frontend prompts re-login without losing the partially rendered answer.
- Personnel entity attributes (names) are redacted in API responses for roles below engineer.

14.6 Testing

Unit tests cover token issuance, expiry rejection, and each route's guard against every role. No penetration testing is in scope for the MVP.

14.7 Dependencies

PyJWT, passlib/bcrypt, the users table (Chapter 7).

14.8 Future extension

Keycloak (OIDC/LDAP) with OPA (ABAC) and mTLS is the named enterprise upgrade (Chapter 19).

Chapter 15

Evaluation

15.1 Philosophy

Every figure presented to a judge must be something the team actually measured on the actual demo corpus. No published benchmark, no inter-annotator agreement study, no ablations -- those are legitimate future work ([Chapter 19](#)), not claims the team can defend under live questioning.

15.2 Evaluation dashboard

Dashboard tile	How measured	Why it's checkable
Knowledge Coverage	% of demo-corpus equipment tags with at least one linked document/fact	Directly visible on the graph itself
Citation Accuracy	Manual spot-check: does the cited passage support the sentence, on 20 answers	Demonstrates the anti-hallucination story concretely
Retrieval Accuracy	15–20 hand-written test queries with a known correct source document, precision@5, including queries specifically covering implicit context and cross-domain bridges (Chapter 9)	Small enough to build and run in an afternoon; a judge can ask a live variant
Retrieval Latency	Simple timing harness, P50/P95 over 30 queries, split by shallow vs. full traversal mode	Shows the system is demo-fast, not just demo-correct
Agent Success Rate	% of scripted agent invocations (Asset history/QA, Asset CGFR, Diagnose, Comply) completing within the iteration cap with an evidence-linked answer	Directly testable live by a judge
Compositional Accuracy	Precision/recall on 5–8 hand-constructed CGFR test queries with a known correct recommendation	Directly testable live by a judge; specifically stresses the “answer was never written down” claim
Compliance Detection	5 deliberately introduced gaps in the demo corpus, precision/recall on those 5	Directly testable live by a judge

Table 15.1: Evaluation dashboard metrics

15.3 Ground truth creation

- Retrieval test set. 15–20 queries hand-written against the actual demo corpus, including at least 3 implicit-context queries (no named entity, relying on chat history or `focused_tag`) and at least 2 cross-domain-bridge queries (relying on structural enrichment), each with the known correct source document(s) recorded before any retrieval code is run against them.
- Citation spot-check set. 20 Copilot answers sampled after the retrieval/generation pipeline is stable; each cited sentence checked by hand against its cited passage.
- Entity-extraction spot-check. 20 documents hand-reviewed for extracted entities/relationships *and specification facts* against what a human reader would extract.
- Compositional test set. 5–8 hand-constructed component-selection questions (modelled on the funnel/pipe example) against a demo corpus deliberately seeded with a spare-parts catalog containing dimensional, material, and rating specifications, with the correct recommendation (or correct “no compatible candidate” verdict) recorded in advance.
- Compliance test gaps. 5 gaps deliberately introduced into the demo corpus.

15.4 Manual evaluation process

One team member (rotating) runs each spot-check independently and records a binary correct/incorrect judgement per item in a shared spreadsheet; disagreements on ambiguous cases are resolved by a second reviewer before the number is finalised for the dashboard.

15.5 Dashboard implementation

The evaluation dashboard is a page in the frontend (Chapter 13), reading its numbers from a small `evaluation_results` table populated by a one-off script run during Week 5, re-run whenever a subsystem changes materially enough to warrant re-measurement.

15.6 Expected targets

Metric	Target
Knowledge Coverage	$\geq 90\%$ of the 15–25 demo equipment tags
Citation Accuracy	$\geq 90\%$ of spot-checked citations support their sentence
Retrieval Accuracy (precision@5)	$\geq 80\%$ on the 15–20 hand-written queries
Retrieval Latency	P95 within the targets in Chapter 4
Agent Success Rate	$\geq 85\%$ of scripted invocations
Compositional Accuracy	$\geq 4/6$ hand-constructed CGFR queries correctly recommended (or correctly flagged as unresolvable)
Compliance Detection	$\geq 4/5$ deliberate gaps correctly flagged, ≤ 1 false positive across the checked set

Table 15.2: Expected evaluation targets

15.7 Dependencies

The seeded demo corpus and its ground-truth annotations, the retrieval and agent modules under test, a small evaluation-runner script (Chapter 16).

15.8 Future extension

A full published-benchmark-style evaluation with inter-annotator agreement and ablation studies is the named roadmap item (Chapter 19).

Chapter 16

Testing

16.1 Testing philosophy

Given a 5-week, 4-person timeline, testing is scoped to what protects the demo: every pipeline stage and every agent has enough automated testing to catch a regression before rehearsal, and the full demo script itself is the top-level acceptance test.

16.2 Unit testing

Area	What is unit-tested
Entity resolution	Exact match, fuzzy match threshold behaviour, embedding fallback trigger condition
Fact/trust model	Supersession transaction (old row marked superseded , new row active , never a delete), for both relational and specification-type predicates
Context assembly	Explicit/implicit tag extraction, focused-node insertion, chat-history mining, capping behaviour (Chapter 9 , §9.4)
Adaptive traversal	Relevance-decay pruning, node-budget enforcement, edge-confidence propagation, best-first ordering (Chapter 9 , §9.6)
Structural enrichment	Bridge-node discovery correctness on a small fixture graph, hub-detection threshold behaviour (Chapter 9 , §9.7)
Retrieval fusion	Correct pathway weights selected per query type; renormalisation when a pathway is empty
Citation resolution	All tags resolve; one tag fails and regenerates; two failures flag the sentence
Agent routing	Supervisor picks the expected worker for each query-type/trigger combination, including compositional/compatibility triggers routing to CORTEX Asset
CGFR reasoning	Constraint extraction on a fixed query set; compatibility filtering on a fixed set of target/candidate spec pairs with known pass/fail outcomes (Chapter 11 , §11.5.4)
Auth	Token issuance/expiry, route guard pass/fail per role

Table 16.1: Unit test coverage by area

16.3 Integration testing

Integration tests run against a seeded, demo-scale instance of Postgres, Neo4j, and Qdrant (brought up via the same Docker Compose file used for the real deployment, with a small fixed fixture corpus of ~10 documents including a spare-parts catalog with specification data):

- Full ingestion pipeline: upload → parse → extract (relational and specification facts) → resolve → facts written → graph materialised → vectors indexed, asserting expected node/edge/vector counts.
- Full Copilot request lifecycle (Section 5.8) against the fixture corpus, asserting a citation-resolved answer is returned within the latency budget, exercised in both shallow and full-traversal modes.
- Each of the three agent workers end-to-end against the fixture corpus, including at least one full CGFR chain against the fixture’s spare-parts catalog with a known expected recommendation.

16.4 End-to-end (UI) testing

A Playwright/Cypress script drives the browser through the exact demo script (Chapter 18): login → dashboard → click P-101A → invoke CORTEX Diagnose → ask CORTEX Asset a compositional “which part fits” question → switch to compliance view → submit a correction → assert the graph updates. Run before every rehearsal as a regression gate.

16.5 AI-specific testing

- Extraction schema conformance. Every fast-tier extraction call’s output is validated against the extended JSON schema (Chapter 10); a fixed set of sample chunks with expected triples -- including specification-type triples -- acts as a regression suite.
- Prompt regression. Representative prompts (Copilot generation, Diagnose hypothesis generation, CGFR constraint extraction and compatibility filtering) are re-run against a small fixed query set whenever a prompt changes, with manual review of the diff in output quality.
- Evaluation dashboard (Chapter 15) doubles as an AI-quality regression check.

16.6 Test data

The ~10-document integration fixture is a fixed, hand-picked subset of the full 100-300 document demo corpus, small enough to run in CI-like fashion in seconds to low minutes, large enough to exercise every entity type, at least one P&ID, and enough specification data (on both equipment and spare parts) to exercise a full CGFR chain.

16.7 What is explicitly not tested

Load testing beyond the P95 latency harness in Chapter 15, security penetration testing, and multi-region failover are out of scope.

16.8 Dependencies

pytest (unit/integration), Playwright or Cypress (end-to-end), the Docker Compose stack ([Chapter 17](#)) for integration fixtures.

Chapter 17

Deployment

17.1 Target

Docker Compose on a single workstation or a single cloud VM. No Kubernetes, no multi-node orchestration.

17.2 Docker Compose services

```
1 version: "3.9"
2 services:
3   postgres:
4     image: postgres:16
5     environment:
6       POSTGRES_DB: cortex
7       POSTGRES_USER: cortex
8       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
9     volumes: ["/data/postgres:/var/lib/postgresql/data", "/migrations:/
10      migrations"]
11     healthcheck:
12       test: ["CMD-SHELL", "pg_isready -U cortex"]
13   neo4j:
14     image: neo4j:5-community
15     environment:
16       NEO4J_AUTH: neo4j/${NEO4J_PASSWORD}
17     volumes: ["/data/neo4j:/data"]
18     ports: ["7474:7474", "7687:7687"]
19
20   qdrant:
21     image: qdrant/qdrant:latest
22     volumes: ["/data/qdrant:/qdrant/storage"]
23     ports: ["6333:6333"]
24
25   redis:
26     image: redis:7
27
28   fabric-api:
29     build: ./backend
30     env_file: .env
31     depends_on: [postgres, neo4j, qdrant, redis]
32     ports: ["8000:8000"]
33
34   ingestion-worker:
35     build: ./backend
36     command: rq worker ingestion
```

```

37     env_file: .env
38     depends_on: [postgres, neo4j, qdrant, redis]
39
40 frontend:
41     build: ./frontend
42     env_file: .env
43     ports: ["3000:3000"]
44     depends_on: [fabric-api]

```

Listing 17.1: docker-compose.yml (representative) – unchanged by context-aware retrieval and compositional reasoning, both of which run inside fabric-api

This is exactly the four deployable units fixed for the MVP: (1) ingestion worker, (2) fabric API, (3) Next.js frontend, (4) Postgres+Neo4j+Qdrant+Redis as managed containers. Context-aware traversal and CGFR compositional reasoning add code inside fabric-api, not a new service.

17.3 Folder structure

```

1  cortex/
2    backend/
3      app/
4        api/           # FastAPI routers: documents, query, entities, agents
5        ingestion/     # Docling parsing, extraction (incl. spec predicates),
6          resolution
7        retrieval/     # chunking, pathways, context assembly, seed expansion,
8          # adaptive traversal, structural enrichment, fusion,
9          reranking
10       agents/        # LangGraph supervisor + asset (incl. CGFR)/diagnose/
11         comply
12       kg/            # Neo4j driver wrapper, Cypher queries
13       db/            # SQLAlchemy models, migrations
14       auth/          # JWT, role guards
15       tests/
16       Dockerfile
17 frontend/
18   app/               # Next.js app router pages
19   components/        # GraphExplorer, CopilotChat, ReasoningOverlay,
20     ComparisonTable, ...
21   lib/               # API client, SSE/WS handling, GraphStore
22   Dockerfile
23 migrations/
24   001_init.sql
25   002_facts_fts.sql
26   003_facts_predicate_index.sql
27 docker-compose.yml
28 .env.example

```

Listing 17.2: Repository layout

17.4 Environment variables

Variable	Purpose
POSTGRES_PASSWORD, NEO4J_PASSWORD	Datastore credentials
DATABASE_URL	Postgres connection string for the fabric API and worker
NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD	Neo4j driver connection
QDRANT_URL	Qdrant connection
REDIS_URL	Job queue connection
JWT_SECRET	Signing key for auth tokens
FAST_MODEL_API_KEY, FRONTIER_MODEL_API_KEY	LLM provider credentials per tier (Chapter 10)
EMBEDDING_MODEL_ENDPOINT	Embedding provider or local model path

Table 17.1: Environment variables (.env)

17.5 Startup sequence

1. `docker compose up -d postgres neo4j qdrant redis` and wait for health checks.
2. Apply migrations: `psql $DATABASE_URL -f migrations/001_init.sql` (and subsequent files in order, including `003_facts_predicate_index.sql`).
3. Seed demo users ([Chapter 14](#)) and the canonical tag/alias list ([Chapter 8](#)) via a one-off seed script.
4. `docker compose up -d fabric-api ingestion-worker frontend`.
5. Bulk-ingest the demo corpus (including a spare-parts catalog with specification data) via `POST /api/documents` (scripted loop) or a one-off ingestion CLI.

17.6 Configuration management

All configuration is environment-variable driven; no hardcoded endpoints or credentials in code.

17.7 Observability

Basic structured logging (JSON lines, one line per pipeline stage/agent step/API request) to stdout, collected via `docker compose logs`; a Postgres `audit_log` table provides the user-facing audit trail, including a per-step log for CGFR chains.

17.8 Degradation plan for the demo

If P&ID live extraction proves unreliable close to demo day, the fallback is to pre-load 2-3 curated drawings' graph edges directly via a seed script. If context-aware traversal's structural-enrichment phase proves slow on stage, it is dropped per the fallback in [Chapter 9](#), and the demo continues on adaptive-traversal results alone. If a CGFR query's candidate gathering finds nothing in the live corpus, the rehearsed fallback is to have the demo corpus seeded with a spare-parts catalog verified in advance to contain a working example.

17.9 Dependencies

Docker, Docker Compose, a single workstation or cloud VM with sufficient resources to run Postgres, Neo4j, Qdrant, Redis, and two Python services plus a Next.js app concurrently

17.10 Future extension

K3s/Kubernetes is the named upgrade trigger for multi-plant scale and independent service scaling ([Chapter 19](#)); not before.

Chapter 18

Demo

18.1 Why one continuous interaction

A demo that is ‘upload docs → chat → graph → RCA → compliance’ is technically complete but forgettable. The fix is choreographing the existing features into one continuous, visual interaction judges watch unfold -- and, critically, the demo must include a moment where CORTEX answers a question that was never written down anywhere in the corpus, because that is the single most differentiated capability of the system.

18.2 Complete demo script — “A Day in the Life”

Beat	Time	What happens
1. Morning alert	0:00–0:45	Graph dashboard shows a rising-vibration pattern on P-101A highlighted, matching a historical pre-failure signature (CORTEX Asset). Speaking point: “The system doesn’t wait for a work order — it noticed this pattern on its own.”
2. Click P-101A	0:45–2:00	Graph animates outward via context-aware traversal: work orders, inspection report, OEM manual, sister pump P-101B all light up; CORTEX Ask streams a cited history summary alongside. Speaking point: name the citation chip, click one to show the source passage. Note that <code>focused_tag</code> is now P-101A for every subsequent question.
3. Invoke CORTEX Diagnose	2:00–4:00	Reasoning steps appear live, each hypothesis linked to a highlighted node; final answer names misalignment as the likely root cause. Speaking point: narrate the self-critique step discarding a weaker hypothesis.
4. Ask a compositional question	4:00–5:30	With P-301 focused on the graph, ask “Which funnel should I use for this pipe?” CORTEX Asset streams its CGFR chain live: decomposing constraints, gathering P-301’s specs and candidate funnels, evaluating compatibility, and recommending F-42 with a full citation chain, rendered as a structured comparison table. Speaking point: “No document in this corpus ever says to use F-42 on P-301 — the system just constructed that answer from four separate specifications.”
5. Compliance score view	5:30–6:30	Switch to 86% score; P-101A’s overdue recertification shown in red; click through to the missing evidence on the graph.
6. Correction → live graph update	6:30–7:30	Engineer types a correction note; it’s written as a new fact and the graph updates live on screen.
7. Close	7:30–8:00	Return to the graph-first dashboard; state the evaluation dashboard numbers (Chapter 15), including the compositional-accuracy figure, as the closing, checkable claim.

Table 18.1: Demo script with timing and speaking points

18.3 Transitions

Every transition between beats is a graph interaction (a click, a worker invocation, a view switch), never a slide change or a context switch to a different tool.

18.4 Judge questions and expected answers

Likely question	Expected answer
“How do you know the answer isn’t hallucinated?”	Every sentence carries a <code>[doc_id:passage_id]</code> tag or a graph <code>fact_id</code> checked against the retrieved set before the answer ships; click any citation to see the exact source passage (Chapter 9).
“How did it know to recommend F-42 when no document ever mentioned it alongside P-301?”	CORTEX Asset’s compositional reasoning mode decomposed the question into specification constraints, gathered P-301’s specs and every candidate funnel’s specs separately, and filtered by compatibility — the recommendation is constructed from four individually-cited facts, not retrieved from a single passage (Chapter 11 , §11.5.4).
“What happens if two documents disagree?”	Both facts remain queryable; the newer one is marked active, the older superseded — nothing is silently overwritten (Chapter 7 , Chapter 8).
“Does this scale past a few hundred documents?”	The MVP is deliberately scoped to the demo envelope; Chapter 19 names the exact upgrade path for each simplification.
“What if no compatible part exists?”	CORTEX Asset says so explicitly and names which constraint is hardest to satisfy, rather than recommending the closest partial match silently (Chapter 11 , §11.5.4).
“Can an agent take an action on its own?”	No auto-execute tier exists; Tier-2 proposals require a role-gated human confirm/dismiss click, and a compositional recommendation is always Tier 1 (informational).
“What’s your actual accuracy?”	Point directly to the live evaluation dashboard numbers (Chapter 15), including the compositional-accuracy figure, all measured on the real demo corpus.

Table 18.2: Anticipated judge questions and rehearsed answers

18.5 Failure recovery during the demo

If this fails live	Recovery
P&ID live VLM extraction is slow/unreliable	Fall back to 2–3 pre-processed drawings with graph edges pre-loaded.
Wi-Fi/network drops mid-demo (LLM API call)	Have a second, pre-recorded run of the same exact script ready to play as a fallback narration.
Copilot answer’s citation fails to resolve on stage	Acknowledge it directly, which is itself proof the safety mechanism works.
Agent exceeds its iteration cap	Show the explicit “inconclusive” response as intended behaviour, not a crash.
CGFR finds zero compatible candidates on the live corpus during rehearsal	Confirm the demo corpus’s seeded spare-parts catalog in advance so the rehearsed query has a verified answer; if it happens live anyway, narrate the explicit “no compatible candidate” response as the correct, honest behaviour rather than a failure.

Table 18.3: Failure recovery playbook

18.6 Rehearsal plan

The full script is rehearsed end-to-end at least three times during Week 5 against the real demo corpus, timed with a stopwatch, with every team member able to deliver every speaking point -- including the compositional-reasoning beat, which is rehearsed separately at least once more given it is the newest and most judge-differentiating part of the script.

Chapter 19

Future Enterprise Roadmap

19.1 Principle

Every simplification made for the MVP has a documented, named upgrade path rather than a dead end. If, while implementing any chapter of this document, a team member finds themselves wanting to add a service, database, agent, or framework not already named in [Chapter 5](#) through [Chapter 18](#), the correct action is to write it into the table below (if it isn't already there) and continue building the MVP as specified -- not to build it now.

19.2 MVP → enterprise upgrade path

MVP choice	Enterprise upgrade	Trigger to invest
Simple facts table	Full bitemporal claim store (TimescaleDB)	Multi-site deployment, regulatory audit-replay requirement
Docker Compose	K3s → full Kubernetes	Multi-plant scale, need for independent service scaling
Direct async calls	Kafka event backbone	Sustained 1,000+ docs/day ingestion
Neo4j single node	Memgraph + analytical mirror	10M+ node graphs, streaming ingestion
Rule-based query router	Learned, offline-trained pathway-weighting classifier	Enough logged query/relevance data to train on
Best-first, relevance-decayed adaptive traversal	Full Neo4j GDS Enterprise (proper community detection, centrality algorithms) replacing the degree-based hub heuristic	Graph scale or precision requirements the degree heuristic can no longer satisfy
Query-time structural enrichment (shortest paths, hub heuristic)	Microsoft GraphRAG-style offline community summarisation	Enough offline compute budget and a static-enough corpus to amortise hours of clustering
CGFR as an in-agent reasoning pattern	A dedicated compatibility-rules engine encoding company-specific selection policy (<code>selectionRule</code> relationship type)	A real customer with proprietary, non-obvious selection rules that a frontier LLM cannot infer zero-shot
Parametric material-compatibility knowledge	A dedicated material-compatibility lookup tool/database queried by the agent instead of relying on LLM training data	Evidence of hallucination on obscure material pairs during real usage
Free-text <code>predicate</code> column	A typed <code>Specification</code> node label with schema-validated properties	Specification volume and query complexity outgrow scalar node/edge properties
VLM-based P&ID extraction	RT-DETR + SAM2 connectivity-graph pipeline	Need for verified, high-precision drawing topology at scale
Two autonomy tiers	Three-tier autonomy with OPA policy engine	Regulated customer requiring formal auto-execute governance
JWT + coarse roles	Keycloak (OIDC/LDAP) + OPA (ABAC)	Enterprise SSO and fine-grained, time-bounded access requirements
Practical multi-metric eval (incl. compositional accuracy)	Full published-benchmark-style evaluation with IAA, ablations, significance testing	Pre-sales technical due diligence, published case study
No OT integration	Historian-replica / data-diode pattern	Customer with a live process historian to integrate

Table 19.1: MVP to enterprise upgrade path

19.3 Sequencing if the project continues past the hackathon

Should CORTEX continue past the hackathon, the recommended order of investment follows the trigger conditions above, roughly: (1) ingestion volume growth triggers the Kafka backbone before anything else, since it is purely additive to the existing pipeline shape; (2) a second real customer triggers Keycloak/OPA before a bitemporal store, since access control is usually the first procurement blocker; (3) a dedicated material-compatible lookup tool and a selectionRule relationship type are the next-cheapest wins, since they directly reduce the residual hallucination risk in compositional reasoning identified in [Chapter 10](#); (4) graph scale, full GDS, and the P&ID vision pipeline are the most expensive upgrades and should wait for clear evidence (node counts, drawing-topology precision requirements) rather than being pre-built speculatively.

19.4 Closing

Every cut recorded in this document removes something a real production platform would eventually need and nothing a hackathon judge can actually evaluate in a live demo. What remains is still a genuinely differentiated product: a knowledge graph built from real industrial documents, hybrid retrieval with honest citations, a context-aware graph traversal pipeline that understands conversational intent rather than just keyword-match entities, a working multi-agent system with three specialist workers that map directly onto the problem statement's own suggested features, a compositional reasoning capability that constructs answers no document ever stated, a P&ID-to-graph pipeline, and an evaluation suite the team can defend under questioning because every number in it was actually measured on the actual demo corpus.

Appendix: Decision Log

Every significant technology and design choice made during the MVP design, with its rationale and the rejected alternative. This is the detailed backing for the one-line summary in [Section 5.2](#), and demonstrates that architectural choices were deliberate, including the choices made when integrating context-aware retrieval and compositional reasoning into the original architecture.

Decision	Alternative	Reason
FastAPI	Flask	Async endpoint support; native Pydantic validation; auto-generated OpenAPI docs
Neo4j	Memgraph	Ecosystem maturity; better Cypher tooling; GDS algorithm library for graph analytics
Docling (IBM)	MinerU / Marker	Per-element bbox provenance required for passage-level citations; stronger table-structure recognition
PaddleOCR	Tesseract / EasyOCR	Materially better F1 on printed technical fonts; Tesseract kept as offline fallback
Qdrant	Milvus / pgvector	Simpler single-binary deployment; native sparse+dense hybrid search; Rust-core performance
LangGraph	CrewAI / AutoGen	Deterministic supervisor-worker orchestration; first-class support for typed tool contracts and state checkpointing
Next.js + React	Vite SPA / Remix	SSR for fast initial load on low-bandwidth plant networks; ISR for dashboard pages; mature ecosystem
Cytoscape.js	D3.js / vis.js	Purpose-built for large graph rendering; better layout algorithms for industrial ontology graphs
PostgreSQL (facts)	MongoDB	Relational integrity for the fact model; JSONB for flexible metadata; pg_trgm for FTS
Redis (task queue)	Celery + RabbitMQ	Single-binary, zero-ops for 4-person team
Docker Compose	Kubernetes / K3s	Right-sized for a single-machine hackathon demo
JWT (self-issued)	Keycloak / Auth0	Zero additional infrastructure for 3 demo roles
Static RRF fusion	Learned re-ranker	Deterministic, no training data needed for the MVP

Decisions specific to context-aware retrieval integration:

Best-first, relevance-decayed traversal	Fixed *1..2 k-hop only	Solves disconnected- relevance, implicit-context, and cross-domain-bridge failure modes (Chapter 9 , Table 9.1) without a new service
Degree-based hub heuristic	Neo4j GDS Enterprise	No GDS licence required at demo scale; roadmap item if graph scale later demands real community detection
Query-time structural enrichment	Microsoft GraphRAG offline summarisation	Avoids hours of offline LLM compute incompatible with a 5-week hackathon
focused_tag as an explicit payload field	Inferring focus purely from chat-history NLP	Trivial, deterministic, and the frontend already tracks the selected node in GraphStore
Reused Qdrant search for seed expansion	A separate node-embedding index	Avoids a second embedding pipeline and a second vector collection

Decisions specific to compositional reasoning integration:

CGFR as a CORTEX Asset reasoning pattern	A dedicated fourth “Compositional” agent	Keeps the supervisor-worker shape at exactly three workers; avoids duplicating tool plumbing and routing logic for a capability that is really just a different prompt branch
Extended predicate set, prompt-level only	New Specification node label with a graph-schema migration	Neo4j is schema-free on relationship types; a prompt change to the extraction LLM was sufficient and required no migration
No fine-tuning for compositional	Fine-tune on internal	Frontier LLMs already handle constraint decomposition and multi-attribute comparison zero-shot; the real bot-